

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KỸ THUẬT GIAO THÔNG



ĐỒ ÁN TỐT NGHIỆP

ĐỀ TÀI:

**THIẾT KẾ CHƯƠNG TRÌNH ĐIỀU KHIỂN VÀ
GIÁM SÁT VẬN HÀNH TỪ XA CHO ROBOT ĐỊA
HÌNH 6X6**

Học kỳ: 252

GVHD: T.S. Trần Đăng Long

SVTH: Nguyễn Phạm Tuấn Khanh

MSSV: 2211488

TP. HCM, ngày 07 tháng 05 năm 2026

ĐẠI HỌC QUỐC GIA TP.HCM CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM

TRƯỜNG ĐẠI HỌC BÁCH KHOA Độc lập - Tự do - Hạnh phúc

KHOA: KỸ THUẬT GIAO THÔNG

NHIỆM VỤ LUẬN VĂN/ ĐỒ ÁN TỐT NGHIỆP

HỌ VÀ TÊN: Nguyễn Phạm Tuấn Khanh

MSSV: 2211488

NGÀNH: Kỹ thuật Ô tô

LỚP: GT22OTO2

1. Đầu đề luận văn/ đồ án tốt nghiệp: Thiết kế Chương trình điều khiển và giám sát vận hành từ xa cho Robot địa hình 6x6.

2. Nhiệm vụ (yêu cầu về nội dung và số liệu ban đầu):

2.1 Mục tiêu:

Chương trình điều khiển và giám sát vận hành từ xa cho Robot địa hình 6x6.

2.2 Yêu cầu kỹ thuật:

- Yêu cầu về chức năng:
 - + Điều khiển tốc độ và hướng, có thể lựa chọn điều khiển qua 1 trong 3 nguồn sau: Tay cầm RC, Giao diện điều khiển WebServer, Tay cầm (ESP-NOW).
 - + Giám sát cấu hình điều khiển, tốc độ 6 bánh xe, trạng thái vận hành như là: tiến, lùi, đứng yên, tiến trái, tiến phải, xoay vòng trái, xoay vòng phải, lùi trái, lùi phải và điện áp Pin nguồn từ xa thông qua WebServer và Tay cầm (ESP-NOW).
 - + Cơ chế an toàn cho xe dừng lại và thông báo người dùng khi mất kết nối tín hiệu điều khiển hiện tại.
- Yêu cầu về công nghệ:
 - + Sử dụng vi điều khiển ESP32 NodeMCU-32S CH340.
 - + Cấu hình ngắt ngoài tại các chân Pin 36, Pin 39 để đo độ rộng xung tín hiệu (1000-2000 μ s) từ RC Receiver thông qua mạch cầu phân áp.
 - + Cấu hình ngắt ngoài tại các chân Pin 34 (S_1), 35 (S_2), 33 (S_3) cho cụm bánh trái và Pin 25 (S_4), 26 (S_5), 21 (S_6) cho cụm bánh phải để đếm xung vuông từ mạch gia công tín hiệu cảm biến Hall của 6 bánh xe.
 - + Cấu hình kênh ADC1 (Kênh CH4) tại chân Pin 32 để nội suy điện áp Pin nguồn từ mạch cầu phân áp.
 - + Cấu hình ngoại vi phân cứng LEDC của ESP32 xuất xung PWM ở tần số cố định 5kHz (độ phân giải 8-bit) trên 6 kênh độc lập. Các kênh này được gán

tương ứng với Pin 27 (T_1), 14 (T_2), 13 (T_3) cho cụm trái và Pin 17 (T_4), 16 (T_5), 5 (T_6) cho cụm phải.

- + Cấu hình ngõ ra Digital Output kỹ thuật số cho các chân Pin 23 (u_{b1}), 18 (u_{b2}) xuất tín hiệu phanh và Pin 22 (u_{d1}), 19 (u_{d2}) xuất tín hiệu đảo chiều qua mạch gia công tín hiệu để kích hoạt chế độ phanh và đảo chiều.
- + Tích hợp giao thức ESP-NOW và giao thức WebSocket để thiết lập mạng lưới truyền thông điều khiển đa luồng. Trong đó, ESP-NOW cho tay cầm vật lý; WebSocket (vận hành trên mạng Wi-Fi nội bộ SoftAP) chịu trách nhiệm truyền tải dữ liệu song công (chuỗi JSON) liên tục giữa vi điều khiển và giao diện Web giám sát.
- Yêu cầu về hiệu quả: Đảm bảo tính thời gian thực, chu kỳ điều khiển ở mức 20 ms (50 Hz) duy trì ổn định, không bị trễ hay gián đoạn bởi các tác vụ.

3. Ngày giao nhiệm vụ:

4. Ngày hoàn thành nhiệm vụ:

5. Họ tên giảng viên hướng dẫn:

Phản hướng dẫn:

1) T.S. Trần Đăng Long

100%

Nội dung và yêu cầu LVTN/ ĐATN đã được thông qua Khoa.

Ngày tháng năm

CHỦ TRÌ NGÀNH

GIẢNG VIÊN HƯỚNG DẪN CHÍNH

(Ký và ghi rõ họ tên)

(Ký và ghi rõ họ tên)

PHẦN DÀNH CHO KHOA:

Người duyệt (chấm sơ bộ):

Đơn vị:

Ngày bảo vệ:

Điểm tổng kết:

Nơi lưu trữ LVTN/ĐATN:

MỤC LỤC NỘI DUNG

ĐÁNH GIÁ CỦA GIÁO VIÊN HƯỚNG DẪN	9
NHẬN XÉT CỦA GIÁO VIÊN PHẢN BIỆN	10
LỜI NÓI ĐẦU	11
LỜI CẢM ƠN.....	12
CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI	13
1.1. Thể loại đề tài	13
1.2. Đối tượng.....	13
1.3. Mục tiêu đề tài	13
1.4. Các yêu cầu kỹ thuật	14
1.5. Ý tưởng thực hiện.....	15
1.6. Các vấn đề kỹ thuật cần giải quyết và giải pháp cho từng vấn đề	15
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	20
2.1. Tổng quan về Kit Wifi BLE ESP32 NodeMCU-32S CH340	20
2.1.1 Giới thiệu chung và Thông số kỹ thuật	20
2.1.2. Ứng dụng cụ thể trong đồ án.....	21
2.2. Hệ điều hành thời gian thực FreeRTOS	21
2.2.1. Khái niệm Hệ điều hành thời gian thực (RTOS)	21
2.2.2. Tổng quan về FreeRTOS trên ESP32	22
2.2.3. Các thành phần cốt lõi của FreeRTOS.....	22
2.2.4. Phương pháp triển khai API FreeRTOS	24
2.3. Lập trình hướng đối tượng (OOP) trong hệ thống nhúng	25
2.3.1. Khái niệm cơ bản.....	25
2.3.2. Bốn tính chất đặc trưng của OOP	25
2.4. Công nghệ truyền thông không dây	26
2.4.1. Giao thức ESP-NOW.....	26
2.4.2 Webserver, WebSocket và định dạng dữ liệu JSON.....	27
CHƯƠNG 3: THIẾT KẾ BỐ TRÍ CHUNG.....	29
3.1. Giới thiệu.....	29
3.2. Thiết kế bố trí chung	30

CHƯƠNG 4: THIẾT KẾ KỸ THUẬT	31
4.1. Thiết kế kỹ thuật phần cứng	31
4.1.1. Sơ đồ đấu dây	31
4.1.2. Mạch cung cấp nguồn.....	32
4.1.3. Khối thu nhận tín hiệu đầu vào	33
4.1.4. Khối xử lý tín hiệu đầu ra	37
4.1.5. Gia công mạch PCB	39
4.2. Thiết kế phần mềm	41
4.2.1. Kiến trúc phần mềm tổng thể và Mô hình hướng đối tượng (OOP)	41
4.2.2. Triển khai Hệ điều hành thời gian thực (FreeRTOS)	43
4.2.3. Khối thu nhận và Phân quyền điều khiển	45
4.2.4. Khối xử lý Động lực học và Máy trạng thái.....	48
4.2.5. Khối Đo lường và Giám sát.....	50
4.2.6. Khối Truyền thông và Giao diện	52
4.3. Kiểm tra thiết kế kỹ thuật.....	53
4.3.1. Đánh giá Hiệu năng Hệ thống	53
4.3.2. Kiểm tra chức năng giám sát điện áp Pin	54
4.3.3. Kiểm tra chức năng giám sát tốc độ động cơ (RPM)	56
CHƯƠNG 5: KẾT LUẬN	58
5.1. Các kết quả đạt được	58
5.2. Thuận lợi và khó khăn.....	58
5.3. Tồn tại và hạn chế	59
5.4. Quá trình đúc kết kinh nghiệm.....	59
5.5. Kiến nghị và Hướng phát triển.....	59

MỤC LỤC HÌNH ẢNH

Hình 1.2.1. Mô hình Robot địa hình 6x6 và các phương thức điều khiển Tay cầm RC, Web, Tay cầm ESP-NOW.....	13
Hình 1.3.1. Sơ đồ bố trí chung hệ thống điều khiển.....	13
Hình 1.6.1. Giảm đồ thời gian hoạt động các tác vụ	16
Hình 1.6.2. Chuỗi công việc hoạt động các tác vụ.....	17
Hình 1.6.3. Phương pháp đo tốc độ	17
Hình 1.6.4. Phương pháp đo điện áp pin	18
Hình 1.6.5. Giao diện Web.....	19
Hình 1.6.6. Giao diện Tay cầm ESP-NOW	19
Hình 2.1.1 Kit Wifi BLE ESP32 NodeMCU-32S CH340 pinout	20
Hình 2.2.1 Khái niệm RTOS	22
Hình 2.2.2 Thực thi Multi-Task trong RTOS	23
Hình 2.2.3 Các trạng thái hoạt động của Task trong RTOS	23
Hình 2.2.4 Thực tế hoạt động của Multi-Task trong RTOS.....	24
Hình 2.2.5 Queue trong RTOS.....	24
Hình 2.4.1 Sơ đồ hoạt động của HTTP	27
Hình 2.4.2 Sơ đồ hoạt động của WebSocket	27
Hình 3.2.1. Sơ đồ bố trí chung của đề tài.....	30
Hình 4.1.1 Sơ đồ đấu dây.....	31
Hình 4.1.2 Sơ đồ nguyên lý khối mạch cung cấp nguồn (DC-DC Convert)	32
Hình 4.1.3 Sơ đồ nguyên lý mạch đo điện áp Pin.....	33
Hình 4.1.4 Sơ đồ nguyên lý mạch nhân xung	35
Hình 4.1.5 Kết quả mô phỏng tín hiệu đầu ra (Xung màu xanh lá) của Mạch nhân xung bằng Proteus.	36
Hình 4.1.6 Tay cầm RC và RC Receiver	36
Hình 4.1.7 Sơ đồ nguyên lý mạch đọc tín hiệu RC	37
Hình 4.1.8 Sơ đồ nguyên lý mạch PWM/DC	38

Hình 4.1.9 Sơ đồ nguyên lý mạch đệm điều khiển phanh và lùi.....	39
Hình 4.1.10 Mạch cung cấp nguồn + Mạch đo điện áp Pin + Mạch đọc tín hiệu RC.....	40
Hình 4.1.11 Mạch PWM/DC + Mạch nhân xung + Mạch đệm điều khiển phanh và lùi	41
Hình 4.2.1. Sơ đồ cấu trúc lớp OOP	42
Hình 4.2.2 Sơ đồ cấu trúc các Task FreeRTOS	44
Hình 4.2.3 Sơ đồ chuyển trạng thái nguồn điều khiển	46
Hình 4.2.4 Lưu đồ giải thuật phân quyền và xử lý mất kết nối.....	46
Hình 4.2.5 Sơ đồ trạng thái hoạt động	49
Hình 4.2.6 Kết quả đo kiểm tra thời gian hoạt động tác vụ.....	54

MỤC LỤC BẢNG

Bảng 4.1.1 Thống kê tài nguyên ESP32	32
--	----

[illegible]

Người hướng dẫn

9

NHẬN XÉT CỦA GIÁO VIÊN PHẢN BIỆN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Tp Hồ Chí Minh, ngày ... tháng ... năm 2026

Người hướng dẫn

(Kí và ghi rõ họ tên)

LỜI NÓI ĐẦU

Hiện nay, việc duy trì các trạm cảm biến cố định tại các khu bảo tồn thiên nhiên đang bộc lộ nhiều hạn chế. Công tác bảo trì định kỳ tiêu tốn chi phí lớn, trong khi dữ liệu thu thập được thường mang tính cục bộ và thiếu tính cập nhật liên tục. Bên cạnh đó, các vấn đề an ninh như nguy cơ hỏa hoạn do con người, tình trạng trộm cắp hay phá hoại tài nguyên rừng đòi hỏi một hệ thống giám sát có tính cơ động cao hơn là các thiết bị tĩnh.

Trước thực trạng đó, việc ứng dụng robot điều khiển từ xa có khả năng di chuyển linh hoạt trong môi trường rừng núi trở thành một giải pháp tối ưu. Để vận hành hiệu quả trong địa hình gồ ghề, dốc đá và phức tạp, yêu cầu tiên quyết đối với robot là khả năng vượt địa hình vượt trội. Trong số các giải pháp cơ khí hiện nay, cơ cấu Rocker-Bogie nổi lên như một thiết kế kinh điển, đảm bảo sự ổn định và khả năng thích nghi cao với các bề mặt không bằng phẳng.

Tuy nhiên, một nền tảng cơ khí ưu việt chỉ thực sự phát huy tối đa sức mạnh khi được dẫn dắt bởi một phần mềm thông minh, chính xác và đáng tin cậy. Việc quản lý đồng bộ 6 động cơ, xử lý chống nhiễu tín hiệu đa nguồn và đảm bảo phản hồi theo thời gian thực (Real-time) trong điều kiện vận hành khắc nghiệt là một bài toán kỹ thuật phức tạp.

Đề tài “***Thiết kế chương trình điều khiển và giám sát vận hành từ xa cho Robot địa hình 6x6***” tập trung nghiên cứu, ứng dụng hệ điều hành thời gian thực (FreeRTOS) và phương pháp Lập trình Hướng đối tượng (OOP) để xây dựng một kiến trúc phần mềm lõi vững chắc. Mục tiêu của đề án không chỉ là đảm bảo khả năng điều khiển xe chính xác qua nhiều phương thức (Tay cầm RC, WebServer, ESP-NOW) với độ trễ cực thấp, mà còn cung cấp một hệ thống giám sát thông số vận hành trực quan, an toàn, góp phần hoàn thiện năng lực thực chiến của mẫu robot này.

LỜI CẢM ƠN

Lời đầu tiên, em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến thầy TS. Trần Đăng Long, người đã trực tiếp hướng dẫn và sát cánh cùng em trong suốt quá trình thực hiện đồ án. Sự định hướng tận tâm cùng những kiến thức uyên thâm và kinh nghiệm dày dặn của thầy không chỉ giúp em tháo gỡ những vướng mắc kỹ thuật, mà còn truyền cảm hứng về tinh thần tự học, sự tự tin để hoàn thành tốt nhiệm vụ được giao.

Em cũng xin bày tỏ lòng tri ân sâu sắc tới thầy Nguyễn Cao Trí (Khoa Khoa học và Kỹ thuật Máy tính – Trường Đại học Bách khoa, ĐHQG-HCM). Với vai trò là khách hàng và nhà tài trợ chính, sự hỗ trợ hoàn toàn về kinh phí cùng những yêu cầu thực tiễn từ thầy đã tạo điều kiện tiên quyết để em chuyên tâm nghiên cứu và hoàn thiện mô hình một cách tốt nhất.

Em xin trân trọng cảm ơn Ban lãnh đạo Khoa Kỹ thuật Giao thông, Trường Đại học Bách khoa – ĐHQG-HCM, đã tạo mọi điều kiện thuận lợi về môi trường học tập và thủ tục hành chính để dự án được triển khai thuận lợi.

Mặc dù đã cố gắng nhưng do hạn chế về kinh nghiệm và kiến thức, em khó tránh khỏi những thiếu sót. Rất mong nhận được sự góp ý, chỉ bảo từ quý Thầy, Cô và các bạn để em có thể hoàn thiện và ứng dụng đề tài tốt hơn trong thực tế.

Xin chân thành cảm ơn!

TP. HCM, ngày ... tháng ... năm 2026

Sinh viên thực hiện

Nguyễn Phạm Tuấn Khanh

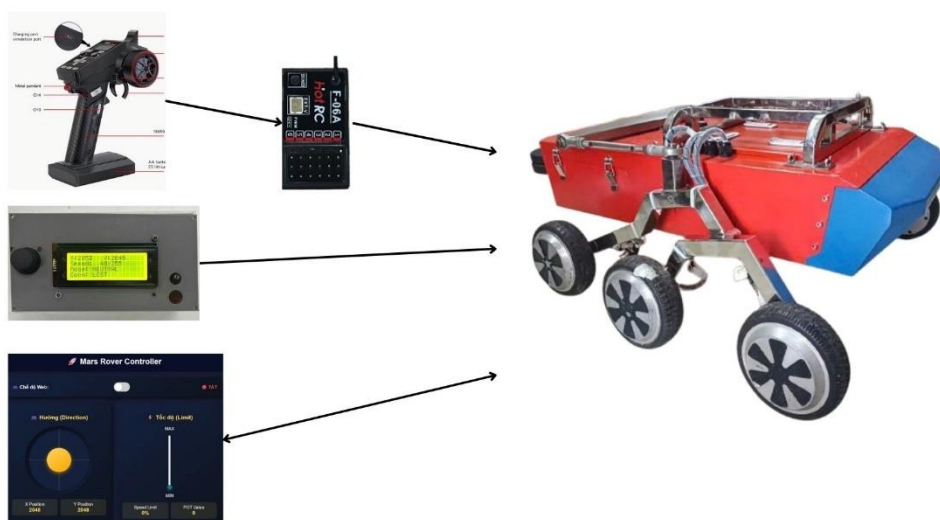
CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI

1.1. Thể loại đề tài

Thiết kế sản phẩm.

1.2. Đối tượng

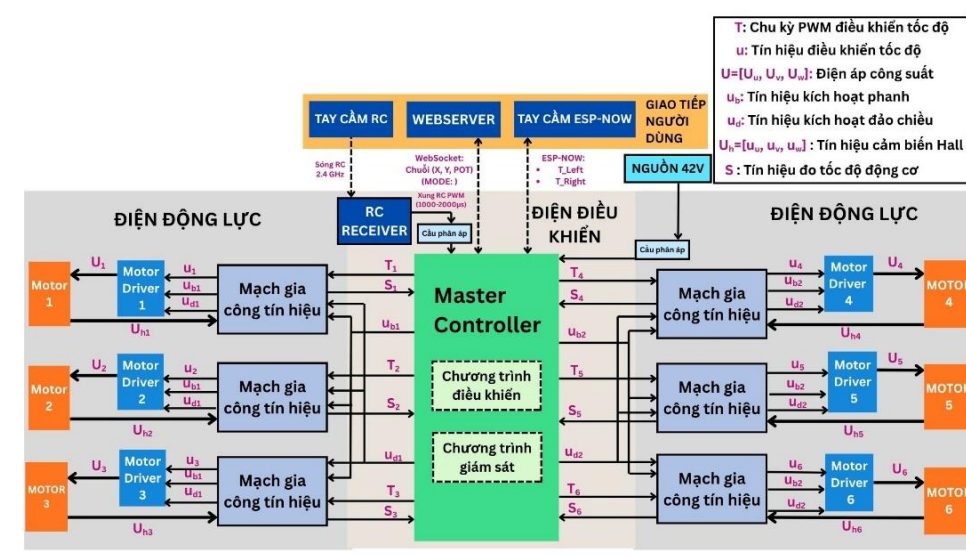
Mô hình Robot địa hình 6x6 sáu bánh dẫn động có tích hợp điều khiển từ xa.



Hình 1.2.1. Mô hình Robot địa hình 6x6 và các phương thức điều khiển Tay cầm RC, Web, Tay cầm ESP-NOW

1.3. Mục tiêu đề tài

Chương trình điều khiển và giám sát vận hành từ xa cho Robot địa hình 6x6.

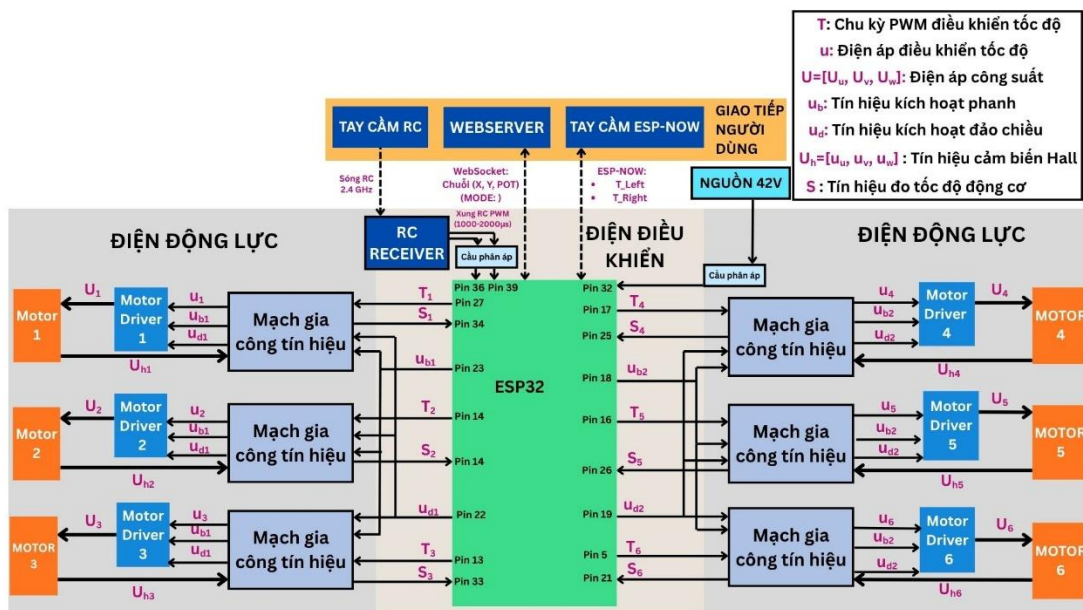


Hình 1.3.1. Sơ đồ bố trí chung hệ thống điều khiển

Ghi chú: Chương trình điều khiển và giám sát vận hành từ xa được triển khai trong khối Master Controller. Chương trình tiếp nhận lệnh lái từ người dùng qua tay cầm RC, Web hoặc ESP-NOW để xử lý phân quyền và máy trạng thái. Từ kết quả đó, ESP32 xuất tín hiệu điều khiển (PWM, phanh, đảo chiều) tới các Motor Driver để vận hành 6 động cơ, đồng thời liên tục đọc tín hiệu hồi tiếp từ cảm biến Hall để giám sát vận tốc, đọc tín hiệu điện áp nguồn Pin để quản lý nguồn Pin và đảm bảo an toàn cho toàn hệ thống.

1.4. Các yêu cầu kỹ thuật

- Chức năng:
 - + Điều khiển tốc độ và hướng, có thể lựa chọn điều khiển qua 1 trong 3 nguồn sau: Tay cầm RC, Giao diện điều khiển WebServer, Tay cầm (ESP-NOW).
 - + Giám sát cấu hình điều khiển, tốc độ 6 bánh xe, trạng thái vận hành như là: tiến, lùi, đứng yên, tiến trái, tiến phải, xoay vòng trái, xoay vòng phải, lùi trái, lùi phải và điện áp Pin nguồn từ xa thông qua WebServer và Tay cầm (ESP-NOW).
- Cơ chế an toàn cho xe sẽ dừng lại và thông báo người dùng khi mất kết nối tín hiệu điều khiển hiện tại.
- Công nghệ:



Hình 1.4.1. Sơ đồ nối dây

- + Sử dụng vi điều khiển ESP32 NodeMCU-32S CH340.
- + Cấu hình ngắt ngoài (chế độ CHANGE) tại các chân Pin 36, Pin 39 để đo độ rộng xung tín hiệu (1000-2000 μ s) từ RC Receiver thông qua mạch cầu phân áp.
- + Thiết lập ngắt ngoài ưu tiên cao tại các chân Pin 34 (S_1), 35 (S_2), 33 (S_3) cho cụm bánh trái và Pin 25 (S_4), 26 (S_5), 21 (S_6) cho cụm bánh phải để đếm xung vuông từ mạch gia công tín hiệu cảm biến Hall của 6 bánh xe.
- + Cấu hình kênh ADC1 (Kênh CH4) tại chân Pin 32 để đo điện áp Pin nguồn thông qua mạch cầu phân áp.
- + Cấu hình ngoại vi phần cứng LEDC của ESP32 xuất xung PWM ở tần số cố định 5kHz (độ phân giải 8-bit) trên 6 kênh độc lập. Các kênh này được gán tương ứng với Pin 27 (T_1), 14 (T_2), 13 (T_3) cho cụm trái và Pin 17 (T_4), 16 (T_5), 5 (T_6) cho cụm phải.
- + Thiết lập chế độ OUTPUT kỹ thuật số cho các chân Pin 23 (u_{b1}), 18 (u_{b2}) xuất tín hiệu phanh và Pin 22 (u_{d1}), 19 (u_{d2}) xuất tín hiệu đảo chiều qua mạch gia công tín hiệu để kích hoạt chế độ phanh và đảo chiều.
- + Tích hợp giao thức ESP-NOW và giao thức WebSocket để thiết lập mạng lưới truyền thông điều khiển đa luồng. Trong đó, ESP-NOW cho tay cầm vật lý; WebSocket (vận hành trên mạng Wi-Fi nội bộ SoftAP) chịu trách nhiệm truyền tải dữ liệu song công (chuỗi JSON) liên tục giữa vi điều khiển và giao diện Web giám sát.
- Hiệu quả: Đảm bảo tính thời gian thực, chu kỳ điều khiển 20 ms (50 Hz) duy trì ổn định, không bị trễ hay gián đoạn bởi các tác vụ.

1.5. Ý tưởng thực hiện

- Sử dụng vi điều khiển ESP32 để xử lý dữ liệu điều khiển và giám sát xe.
- Thiết kế giao diện WebServer để hiển thị cấu hình điều khiển, tốc độ 6 bánh xe, trạng thái vận hành.

1.6. Các vấn đề kĩ thuật cần giải quyết và giải pháp cho từng vấn đề

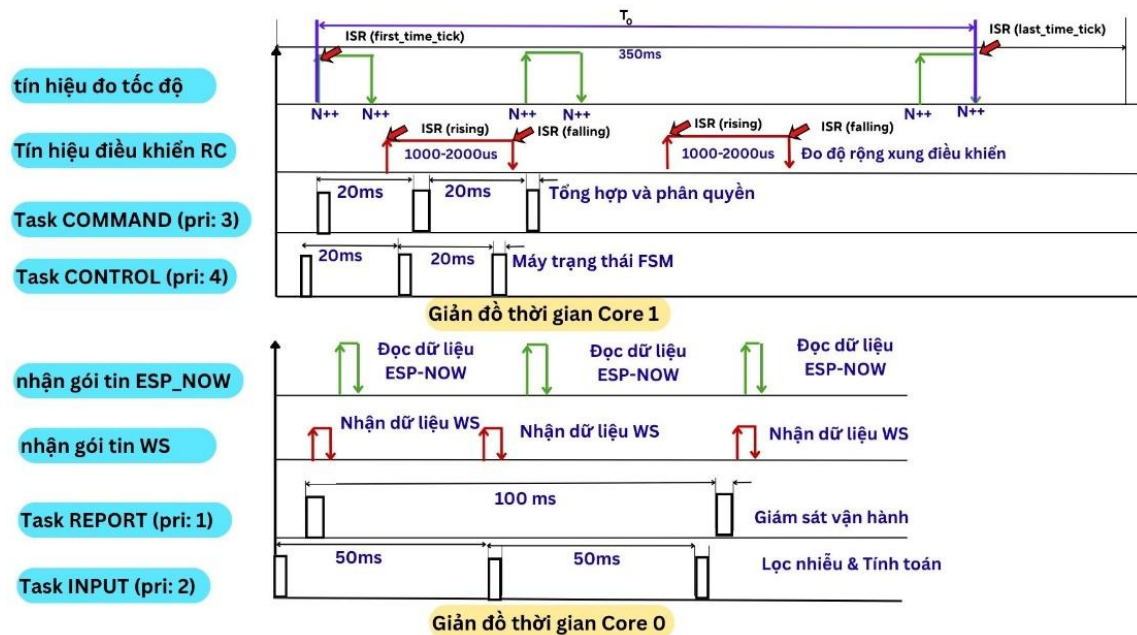
Vấn đề kĩ thuật chính của đề tài:

Vấn đề 1: Đảm bảo tính thời gian thực, chu kỳ điều khiển 20 ms (50 Hz) duy trì ổn

định, không bị trễ hay gián đoạn bởi các tác vụ.

Ý tưởng/Phương pháp giải quyết vấn đề:

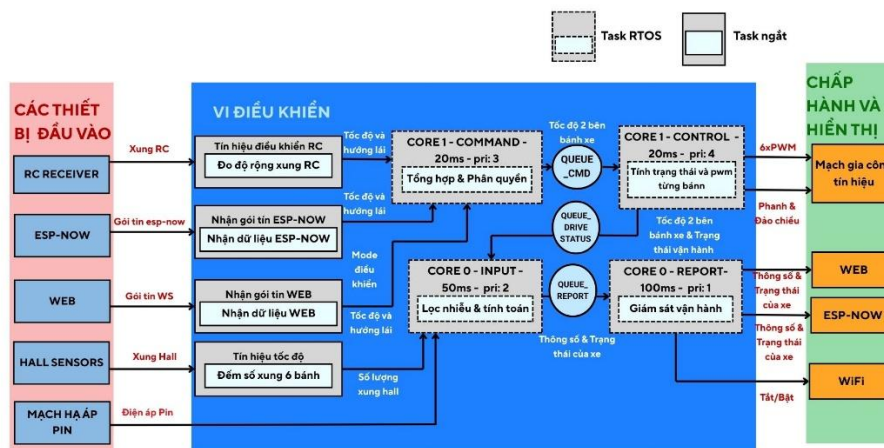
- Sử dụng hệ điều hành FreeRTOS phân bổ 4 tác vụ độc lập trên 2 core xử lý của ESP32 (giao tiếp qua cơ chế hàng đợi) nhằm đảm bảo tính thời gian thực và chu kỳ điều khiển 20 ms (50 Hz) duy trì ổn định. Cụ thể, để hệ thống không bị trễ hay gián đoạn bởi các tác vụ tốn nhiều thời gian (xử lý chuỗi JSON, gửi nhận WiFi), các tác vụ này được cô lập hoàn toàn sang Core 0. Core 1 được dành riêng để thực thi các tác vụ khẩn khe nhằm duy trì chính xác chu kỳ điều khiển 20 ms, thông qua giản đồ thời gian được thể hiện như sau:



Hình 1.6.1. Giản đồ thời gian hoạt động các tác vụ

Ghi chú: Giản đồ thời gian thể hiện chu kỳ, thời điểm kích hoạt, và mức độ ưu tiên của 4 tác vụ cùng các ngắt phần cứng (ISR) trên 2 lõi xử lý, qua đó chứng minh bằng đồ thị việc vòng lặp điều khiển cơ khí (ở Core 1) luôn được kích hoạt tại mốc 20ms.

- Tổ chức công việc của các tác vụ và Ngắt phần cứng nhằm mục đích đưa các tác vụ truyền thông mạng nặng nề khỏi luồng điều khiển động lực học, từ đó bảo vệ tuyệt đối chu kỳ thời gian thực 20ms của xe mà không bị trễ hay gián đoạn.



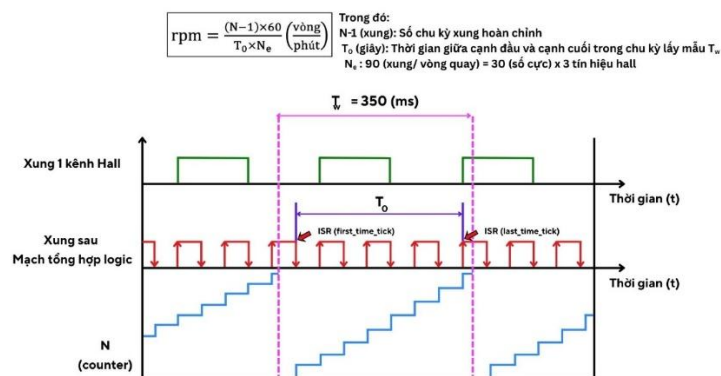
Hình 1.6.2. Chuỗi công việc hoạt động các tác vụ

Ghi chú: Sơ đồ minh họa luồng công việc, dữ liệu giao tiếp giữa 4 tác vụ (Task) và Ngắt phần cứng (ISR) trên 2 lõi ESP32. Trong đó, Core 1 xử lý thời gian thực với Task COMMAND (tổng hợp và phân quyền lệnh lái) và Task CONTROL (điều khiển FSM và xuất PWM). Core 0 xử lý dữ liệu nền với Task INPUT (đo lường cảm biến) và Task REPORT (truyền thông mạng). Các sự kiện ngoại vi siêu tốc (đo xung RC, đếm xung Hall) được tách riêng cho các Ngắt phần cứng (ISR) với quyền ưu tiên tuyệt đối để đảm bảo không bỏ lỡ dữ liệu.

Vấn đề 2: Giám sát cấu hình điều khiển, tốc độ 6 bánh xe, trạng thái vận hành và điện áp Pin nguồn.

Ý tưởng/Phương pháp giải quyết vấn đề:

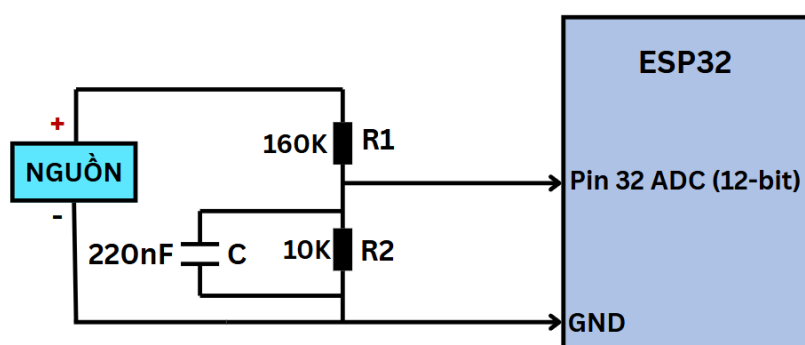
- Phương pháp đo tốc độ các bánh xe bằng cách kết hợp đếm xung và đo thời gian để giải quyết vấn đề giám sát tốc độ các bánh xe.



Hình 1.6.3. Phương pháp đo tốc độ

Ghi chú: Sơ đồ minh họa phương pháp đo tốc độ (M/T), kết hợp đếm số chu kỳ xung hoàn chỉnh (N-1) và đo lường chính xác khoảng thời gian (T0) giữa các sườn xung trong một chu kỳ lấy mẫu cố định ($T_w = 350\text{ ms}$). Để khắc phục sai số ở vận tốc thấp, tín hiệu từ 3 cảm biến Hall được đưa qua mạch tổng hợp logic để nhân 3 độ phân giải ($N_e = 90\text{ xung/vòng}$), giúp vi điều khiển tính toán và phản hồi vận tốc (RPM) một cách mượt mà và chính xác tuyệt đối.

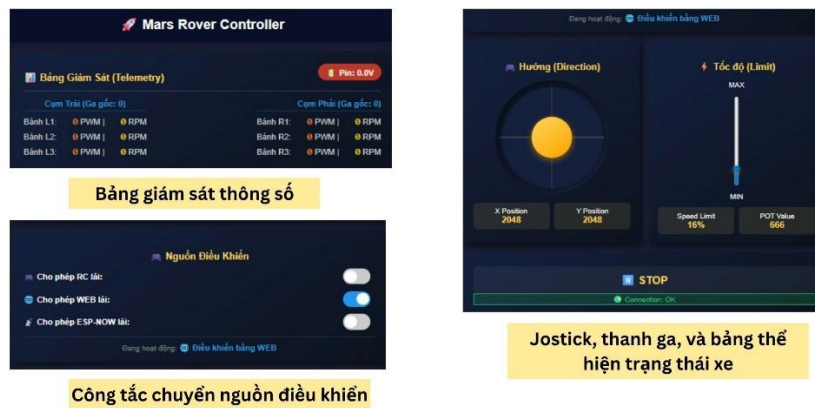
- Phương pháp đo điện áp Pin nguồn bằng cách sử dụng chức năng đọc ADC của ESP32 kết hợp mạch cầu phân áp và tụ lọc phẳng tín hiệu để giải quyết vấn đề giám sát điện áp Pin nguồn.



Hình 1.6.4. Phương pháp đo điện áp pin

Ghi chú: Sơ đồ minh họa phương pháp đo lường và giám sát điện áp nguồn Pin 42V. Hệ thống sử dụng phần cứng mạch cầu phân áp (điện trở 160kΩ và 10kΩ) để hạ điện áp xuống ngưỡng an toàn (<3.3V), kết hợp với tụ điện (220nF) đóng vai trò bộ lọc thông thấp giúp triệt tiêu nhiễu cao tần. Tín hiệu Analog tuyến tính sau đó được đọc bởi bộ chuyển đổi ADC của ESP32 để phần mềm nội suy và tính dung lượng pin thực tế.

- Xây dựng giao diện hiển thị các thông số giám sát cấu hình điều khiển, tốc độ 6 bánh xe, trạng thái vận hành và điện áp Pin nguồn trên WebServer và tay cầm ESP-NOW.



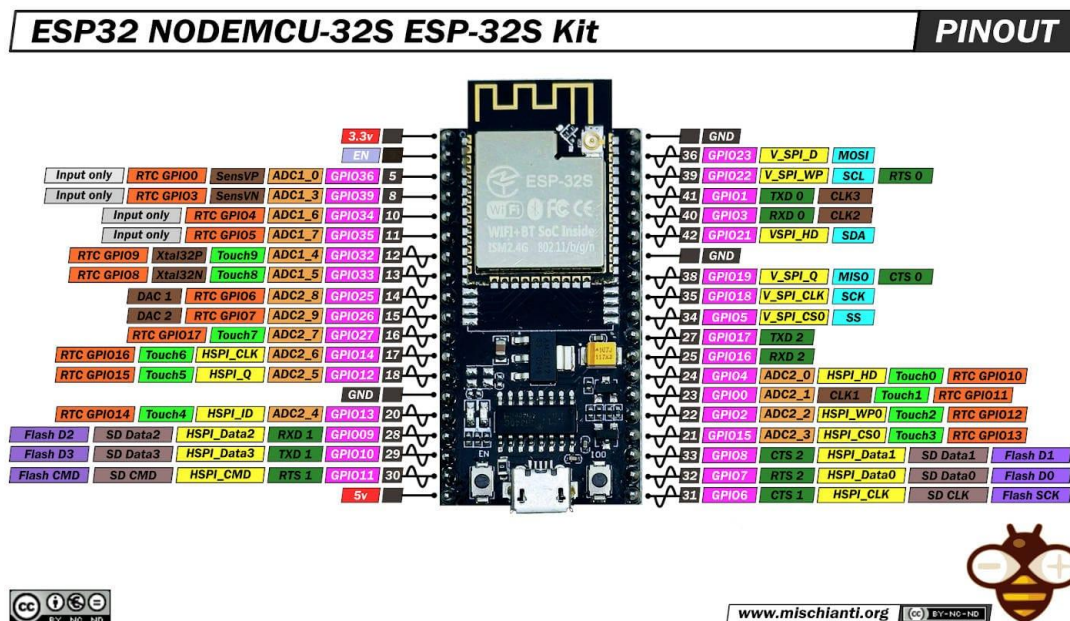
Hình 1.6.5. Giao diện Web



Hình 1.6.6. Giao diện Tay cầm ESP-NOW

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về Kit Wifi BLE ESP32 NodeMCU-32S CH340



Hình 2.1.1 Kit Wifi BLE ESP32 NodeMCU-32S CH340 pinout

2.1.1 Giới thiệu chung và Thông số kỹ thuật

Kit Wifi BLE ESP32 NodeMCU-32S CH340 (được phát triển bởi Ai-Thinker) là một nền tảng vi điều khiển SoC (System on Chip) chi phí thấp, tiêu thụ năng lượng thấp nhưng sở hữu hiệu năng mạnh mẽ. Kit có thiết kế ra chân đầy đủ (38 chân chuẩn 2.54mm), tích hợp sẵn mạch nạp và giao tiếp USB-to-UART CH340, mang lại sự tối ưu cho việc nghiên cứu và phát triển các hệ thống điều khiển IoT.

Thông số kỹ thuật:

- Module trung tâm: ESP32-S
- CPU: lõi kép Xtensa LX6
- SPI Flash: 32Mbits
- Frequency Range: 2400~2483.5Mhz
- Bluetooth: BLE 4.2 BR/EDR
- Wifi: 802.11 b/g/n/e/i
- Support Interface: UART/SPI/SDIO/I2C/PWM/I2S/IR/ADC/DAC
- Nguồn sử dụng: 5VDC từ cổng Micro USB.
- Tích hợp mạch nạp và giao tiếp UART CH340

- Chuẩn 38 chân cắm 2.54mm, ra chân đầy đủ module ESP32
- Tích hợp Led Status, nút nhấn ICO(BOOT) và ENABLE
- Kích thước 25.4 x 48.3mm

2.1.2. Ứng dụng cụ thể trong đồ án

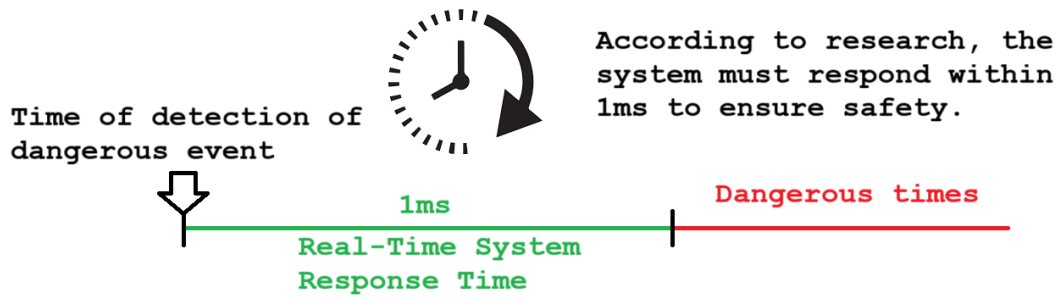
Đối với hệ thống điều khiển robot địa hình 6x6, Kit ESP32 NodeMCU-32S không chỉ đóng vai trò là một vi mạch giao tiếp mạng thông thường mà được khai thác tối đa sức mạnh phần cứng làm Khối xử lý trung tâm (Master Controller). Các đặc tính kỹ thuật được ứng dụng trực tiếp vào đồ án bao gồm:

- Kiến trúc xử lý đa nhân (Dual-Core): Sử dụng 2 nhân xử lý để triển khai hệ điều hành thời gian thực FreeRTOS. Hệ thống phân chia: Core 0 chuyên trách duy trì các giao thức mạng phức tạp (WebServer, ESP-NOW), Core 1 được xử lý tính toán động học và xuất xung điều khiển động cơ.
- Bộ điều khiển PWM (LEDC): Tính năng PWM bằng phần cứng của ESP32 để cung cấp 6 kênh xung PWM độc lập (hoạt động ở tần số 5kHz, độ phân giải 8-bit). Đặc điểm này giúp điều khiển tốc độ cho 6 động cơ BLDC.
- Ngắt ngoài (External Interrupt): Ngoại vi ngắt được thiết lập trên các chân GPIO nhằm đáp ứng tức thời với các sự kiện phần cứng. Tính năng này được ứng dụng để bắt chính xác độ rộng xung từ bộ thu RC (RC Receiver) và đếm xung tốc độ từ 6 cảm biến Hall với độ phân giải micro-giây.
- Bộ chuyển đổi tương tự - số (ADC): Khai thác bộ ADC1 (kênh CH4) để đo giá trị điện áp pin (từ 42V qua cầu phân áp).

2.2. Hệ điều hành thời gian thực FreeRTOS

2.2.1. Khái niệm Hệ điều hành thời gian thực (RTOS)

RTOS - viết tắt của Real-time Operating System hay hệ điều hành thời gian thực, sử dụng trong những ứng dụng yêu cầu thời gian đáp ứng nhanh, chính xác về thời gian. Hai chữ "Real-time" hay "thời gian thực" mang ý nghĩa là thời gian đáp ứng nhanh, đúng như người dùng mong muốn (khác với từ "ngay lập tức").



Hình 2.2.1 Khái niệm RTOS

2.2.2. Tổng quan về FreeRTOS trên ESP32

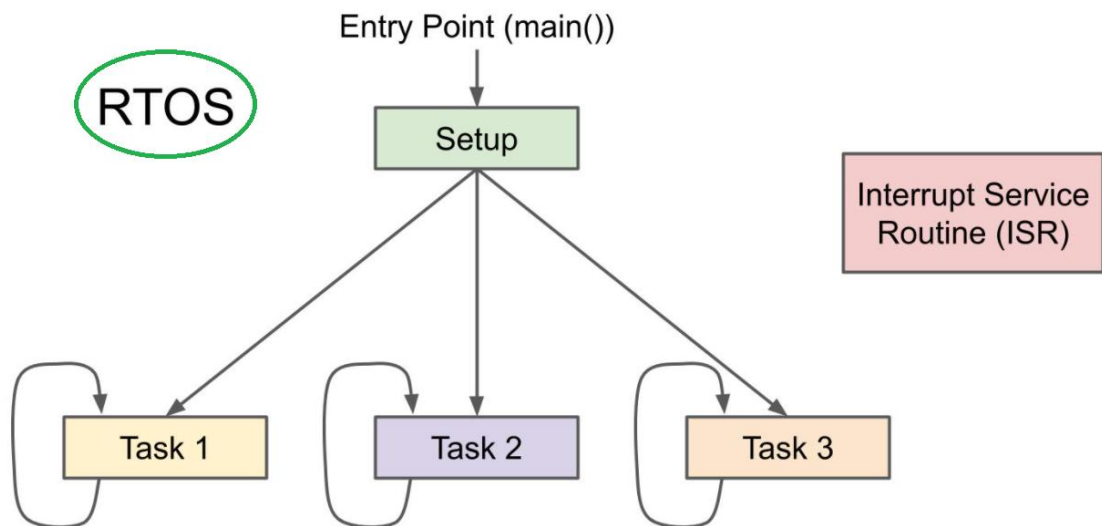
FreeRTOS là một trong những hệ điều hành thời gian thực mã nguồn mở phổ biến nhất thế giới dành cho vi điều khiển. Đối với ESP32, nhà phát triển (Espressif) đã tích hợp sẵn và tùy biến lại nhân FreeRTOS nguyên bản để hỗ trợ Kiến trúc đa xử lý đối xứng. Điều này cho phép FreeRTOS trên ESP32 quản lý và phân bổ các tác vụ chạy song song trên cả hai nhân xử lý (Core 0 và Core 1) một cách đồng nhất, giúp khai thác tối đa sức mạnh phần cứng mà không cần cài đặt thêm thư viện từ bên thứ ba.

2.2.3. Các thành phần cốt lõi của FreeRTOS

Để xây dựng một kiến trúc phần mềm vận hành trơn tru, đồ án khai thác hai cơ chế cốt lõi nhất của FreeRTOS: Tác vụ (Task) và Hàng đợi (Queue).

a. Tác vụ (Task) và Bộ lập lịch (Scheduler)

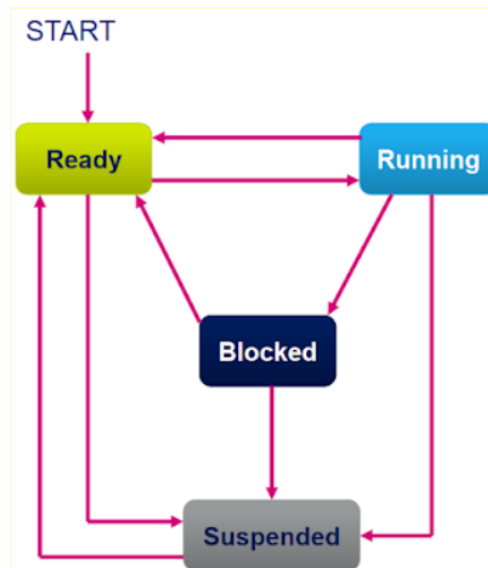
Trong FreeRTOS, chương trình không chạy trong một vòng lặp tuần tự duy nhất mà được chia nhỏ thành các "Tác vụ" (Task) độc lập. Mỗi Task sở hữu không gian ngăn xếp (Stack) và mức độ ưu tiên riêng. Bộ lập lịch (Scheduler) hoạt động theo cơ chế Lập lịch ưu tiên độc chiếm, luôn ưu tiên cấp quyền CPU cho Task có mức ưu tiên cao nhất đang ở trạng thái sẵn sàng.



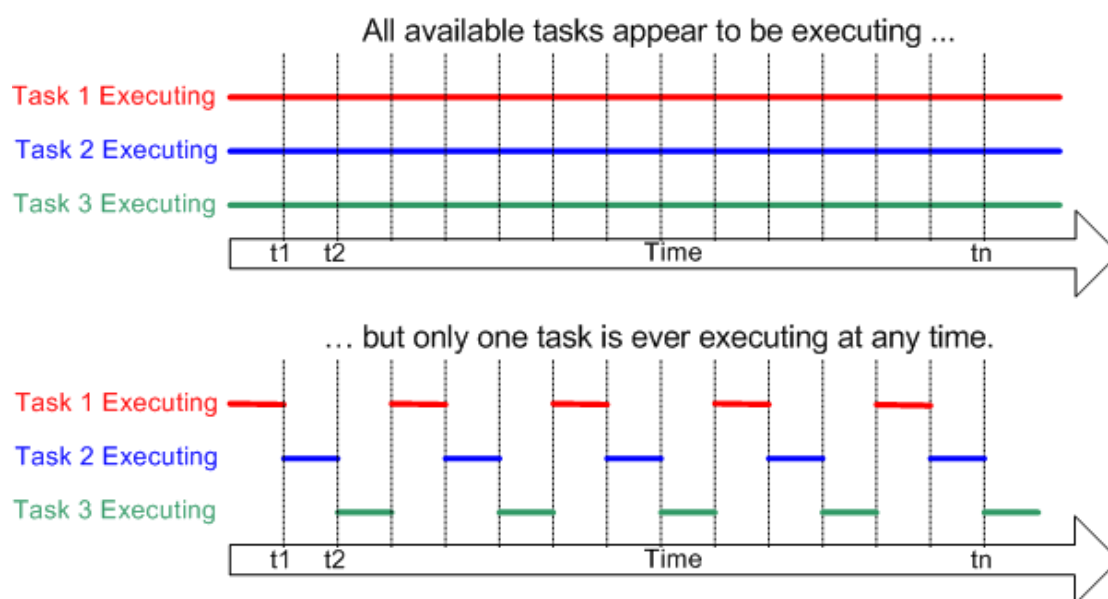
Hình 2.2.2 Thực thi Multi-Task trong RTOS

Một Task trong FreeRTOS luôn tồn tại ở một trong 4 trạng thái:

- Running (Đang chạy): Task đang trực tiếp chiếm dụng CPU để thực thi lệnh.
- Ready (Sẵn sàng): Task đã sẵn sàng nhưng phải chờ CPU đang xử lý Task có ưu tiên bằng hoặc cao hơn.
- Blocked (Bị chặn/Chờ): Task đang tạm dừng chờ sự kiện (hết thời gian delay, có dữ liệu từ Queue, ngắt...). Task ở trạng thái này không tiêu tốn tài nguyên CPU.
- Suspended (Đình chỉ): Task bị tạm dừng hoàn toàn bởi lệnh điều khiển.



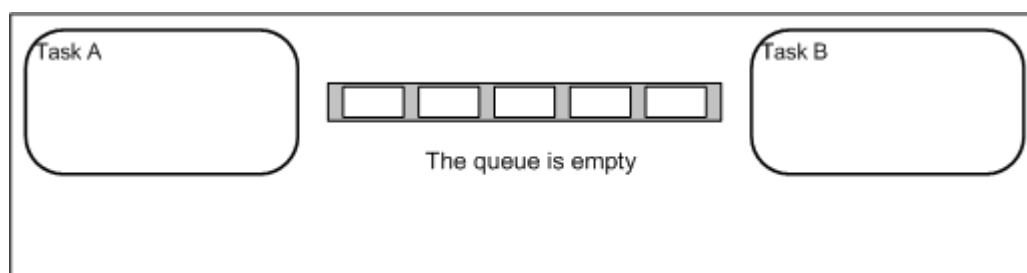
Hình 2.2.3 Các trạng thái hoạt động của Task trong RTOS



Hình 2.2.4 Thực tế hoạt động của Multi-Task trong RTOS

b. Hàng đợi (Queue) – Giao tiếp giữa các tác vụ

Khi các Task chạy song song trên kiến trúc đa nhân, việc chia sẻ biến toàn cục dễ dẫn đến hiện tượng tranh chấp tài nguyên. Để giải quyết, FreeRTOS cung cấp cơ chế Hàng đợi (Queue) hoạt động theo nguyên tắc FIFO (Vào trước - Ra trước). Nó đóng vai trò như một đường ống dữ liệu an toàn (Thread-safe), giúp các Task ở các nhân xử lý khác nhau có thể truyền nhận dữ liệu một cách toàn vẹn.



Hình 2.2.5 Queue trong RTOS

2.2.4. Phương pháp triển khai API FreeRTOS

Hệ thống điều khiển Robot 6x6 được thiết kế đa nhiệm hoàn toàn thông qua việc gọi trực tiếp các API (Application Programming Interface) của FreeRTOS tích hợp sẵn trong nền tảng ESP-IDF:

- Tạo lập và Phân bổ tác vụ: Sử dụng API `xTaskCreatePinnedToCore()` để tách biệt luồng xử lý. Phần giao tiếp mạng (giao thức WiFi/ESP-NOW đòi hỏi thời gian chờ dài) Core 0, Core 1 tính toán Động lực học và điều khiển phản cứng.

- Khóa cứng chu kỳ thời gian thực: Ứng dụng API `vTaskDelayUntil()` thay cho các hàm trễ thông thường. Hàm này tính toán bù trừ thời gian thực thi lệnh, khóa chặt chu kỳ vòng lặp cơ khí ở mức chính xác 20ms (50Hz).
- Quản lý luồng dữ liệu an toàn: Triển khai các API quản lý Hàng đợi như `xQueueCreate()`, `xQueueOverwrite()` và `xQueueReceive()`. Các queue (`QUEUE_CMD`, `QUEUE_DRIVE_STATUS`) làm cầu nối truyền lệnh điều khiển và gói tin giám sát giữa Core 0 và Core 1 mà không xảy ra hiện tượng treo hệ thống (Deadlock).
- Bảo vệ tài nguyên thiết yếu: Sử dụng API `portENTER_CRITICAL_ISR()` trong các hàm phục vụ ngắt (như đếm xung cảm biến Hall) nhằm khóa tạm thời bộ lập lịch, đảm bảo dữ liệu đo lường không bị sai lệch do sự can thiệp của các tác vụ khác.

2.3. Lập trình hướng đối tượng (OOP) trong hệ thống nhúng

2.3.1. Khái niệm cơ bản

Lập trình hướng đối tượng (OOP - Object-Oriented Programming) là một phương pháp luận lập trình dựa trên khái niệm về "Đối tượng" (Object) và "Lớp" (Class). Thay vì viết mã nguồn theo cấu trúc tuần tự tuyến tính truyền thống (Procedural Programming) với các hàm và biến toàn cục rời rạc, OOP chia nhỏ chương trình thành các thực thể phần mềm độc lập.

- Lớp (Class): Là một bản định nghĩa các đặc tính chung.
- Đối tượng (Object): Là một thực thể cụ thể được tạo ra từ Lớp. Mỗi đối tượng chứa cả Dữ liệu (Thuộc tính/Biến) và Hành vi (Phương thức/Hàm) của riêng nó để tự thao tác trên dữ liệu đó.

Trong lĩnh vực lập trình nhúng cho vi điều khiển (sử dụng ngôn ngữ C++), việc ứng dụng OOP giúp quản lý hiệu quả các hệ thống có quy mô mã nguồn lớn, giảm thiểu rủi ro xung đột dữ liệu, đồng thời tăng cường khả năng tái sử dụng (Reusability) và dễ dàng bảo trì (Maintainability).

2.3.2. Bốn tính chất đặc trưng của OOP

a. Tính đóng gói (Encapsulation):

Tính chất này cho phép gom nhóm các thuộc tính và phương thức có liên quan vào cùng một Lớp, đồng thời che giấu trạng thái hoạt động bên trong của đối tượng khỏi sự can thiệp từ bên ngoài thông qua các từ khóa phạm vi truy cập (private, protected, public). Trong hệ thống nhúng, đóng gói giúp bảo vệ các thông số nhạy cảm (như số thứ tự chân GPIO, thanh ghi phần cứng) không bị ghi đè nhầm bởi các module khác, đảm bảo an toàn tuyệt đối cho phần cứng.

b. Tính trừu tượng (Abstraction):

Trừu tượng hóa là quá trình loại bỏ các chi tiết kỹ thuật phức tạp bên dưới, chỉ cung cấp cho người sử dụng những giao diện (Interface) cần thiết nhất. Đối với vi điều khiển, trừu tượng hóa cho phép lập trình viên điều khiển thiết bị (như khởi tạo Timer, cài đặt duty cycle cho PWM) thông qua những hàm cấp cao đơn giản (như `setSpeed()`), mà không cần phải can thiệp trực tiếp vào từng bit của thanh ghi vi xử lý.

c. Tính kế thừa (Inheritance):

Kế thừa cho phép một Lớp mới (Lớp con) được tạo ra dựa trên các thuộc tính và phương thức của một Lớp đã tồn tại (Lớp cha). Tính chất này cực kỳ hữu ích trong việc xây dựng các thư viện phần cứng. Ví dụ: từ một lớp Motor cơ bản, ta có thể kế thừa để tạo ra các lớp `BldcMotor` (động cơ không chổi than) hay `ServoMotor` mà không cần viết lại từ đầu các hàm khởi tạo cấu hình chân cơ bản.

d. Tính đa hình (Polymorphism):

Đa hình cho phép các đối tượng thuộc các lớp khác nhau có thể thực thi cùng một phương thức theo những cách thức riêng biệt của chúng. Trong C++, điều này thường được thực hiện qua hàm ảo (Virtual Function). Nó cho phép hệ thống gọi chung một lệnh điều khiển nhưng mỗi thiết bị ngoại vi sẽ có cách đáp ứng vật lý khác nhau tùy thuộc vào định nghĩa của lớp đó.

2.4. Công nghệ truyền thông không dây

2.4.1. Giao thức ESP-NOW

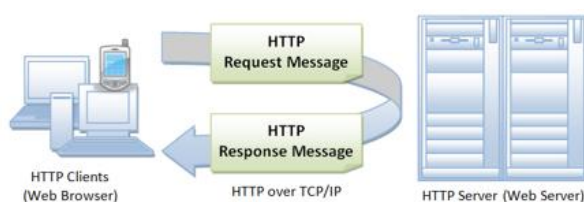
ESP-NOW là một giao thức truyền thông không dây nội bộ do Espressif phát triển, hoạt động dựa trên nền tảng sóng vô tuyến (băng tần 2.4GHz) nhưng loại bỏ hoàn toàn các thủ tục bắt tay (handshake) phức tạp của mạng Wi-Fi tiêu chuẩn. Khác với Wi-Fi truyền thống yêu cầu thiết bị phải kết nối thông qua một Trạm định tuyến trung gian

(Access Point / Router), ESP-NOW cho phép các vi điều khiển ESP32 giao tiếp ngang hàng (Peer-to-Peer) trực tiếp với nhau thông qua địa chỉ vật lý (MAC Address).

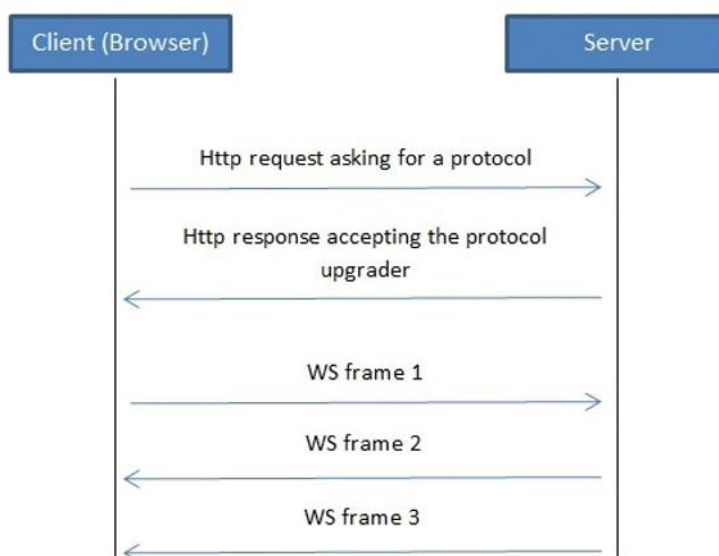
2.4.2 Webserver, WebSocket và định dạng dữ liệu JSON

Để xây dựng một trạm điều khiển từ xa đa nền tảng (truy cập được trên cả điện thoại và máy tính), hệ thống tích hợp chuỗi 3 công nghệ phần mềm:

- WebServer (Máy chủ Web nội bộ): ESP32 được cấu hình phát sóng Wi-Fi độc lập (SoftAP Mode). Nó lưu trữ toàn bộ giao diện điều khiển (HTML, CSS, JS) bên trong phân vùng bộ nhớ Flash (SPIFFS) và trực tiếp "phục vụ" (serve) trang web cho thiết bị người dùng mà không cần kết nối Internet.
- Giao thức WebSocket: Nếu giao thức HTTP truyền thống chỉ hoạt động theo cơ chế "Hỏi - Đáp" (Client gửi Request, Server trả Response rồi ngắt kết nối), thì WebSocket lại tạo ra một đường ống truyền tải dữ liệu song công (Full-duplex) được duy trì liên tục. Chiều gửi và chiều nhận có thể diễn ra đồng thời mà không cần tải lại (reload) trang web.



Hình 2.4.1 Sơ đồ hoạt động của HTTP



Hình 2.4.2 Sơ đồ hoạt động của WebSocket

- Định dạng dữ liệu JSON (JavaScript Object Notation): Là một định dạng chuẩn mở siêu nhẹ, dùng để cấu trúc hóa dữ liệu thành các cặp "Khóa - Giá trị" (Key-Value), rất dễ dàng để máy tính phân tích cú pháp.

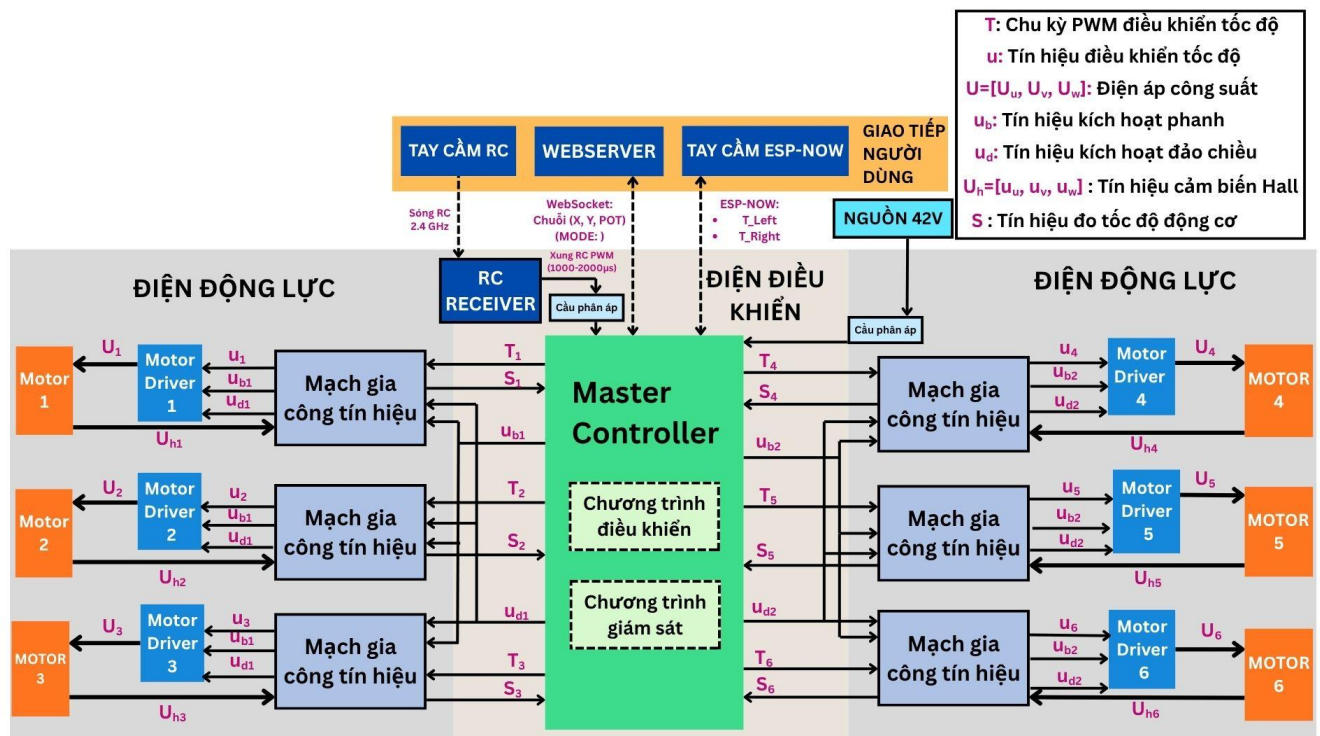
CHƯƠNG 3: THIẾT KẾ BỐ TRÍ CHUNG

3.1. Giới thiệu

Với đề tài này, hệ thống được thiết kế là một mô hình điều khiển và giám sát toàn diện cho Robot địa hình 6x6, mô tả chi tiết luồng truyền thông tín hiệu khép kín từ người dùng đến cơ cấu chấp hành và ngược lại, cụ thể như sau:

- Thu nhận tín hiệu: Hệ thống tín hiệu điều khiển tốc độ và hướng lái từ 3 nguồn điều khiển: Tay cầm RC (thông qua sóng vô tuyến 2.4GHz), WebServer (thông qua mạng WiFi nội bộ bằng giao thức WebSocket) và Tay cầm ESP-NOW (thông qua giao thức ESP-NOW). Các tín hiệu này được truyền về Vi điều khiển trung tâm (Master Controller ESP32) để thực hiện thuật toán phân quyền an toàn và giải mã tọa độ.
- Xử lý và Điều khiển động lực: Xử lý thuật toán lái và điều khiển động cơ bên trong ESP32, sau đó xuất các tín hiệu chu kỳ PWM điều khiển tốc độ ($T_1 \dots T_6$) cùng tín hiệu logic kích hoạt phanh (u_{b1}, u_{b2}), và kích hoạt đảo chiều (u_{d1}, u_{d2}) gửi đến Mạch gia công tín hiệu. Tại đây, tín hiệu được mạch đệm xử lý thành điện áp điều khiển tốc độ analog tuyến tính ($u_1 \dots u_6$) và điện áp kích hoạt (u_b, u_d), tiếp tục truyền đến 6 Motor Driver để phân phối dòng điện công suất 3 pha ($U_1 \dots U_6$), với tổ hợp $U = [U_u, U_v, U_w]$ điều khiển 6 động cơ BLDC.
- Hồi tiếp và Giám sát: Hệ thống liên tục thu thập thông số phản hồi vận hành bao gồm: tín hiệu đo tốc độ động cơ ($S_1 \dots S_6$) được gia công từ tín hiệu cảm biến Hall ($U_{h1} \dots U_{h6}$), với tổ hợp $U_h = [u_u, u_v, u_w]$ của 6 động cơ, và tín hiệu điện áp từ nguồn Pin 42V thông qua mạch cầu phân áp bằng các chân ngắt ngoài và bộ ADC của ESP32, sau đó được tổng hợp, đóng gói thành chuẩn chuỗi JSON để gửi ngược lên giao diện giám sát WebServer và truyền về màn hình LCD của tay cầm ESP-NOW.

3.2. Thiết kế bố trí chung

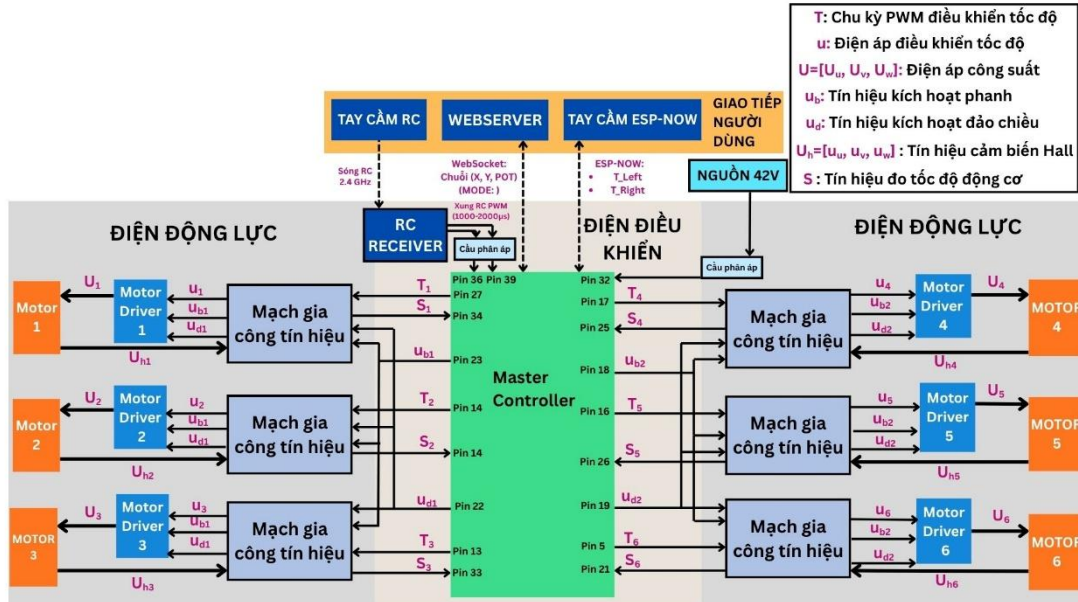


Hình 3.2.1. Sơ đồ bố trí chung của đề tài

CHƯƠNG 4: THIẾT KẾ KỸ THUẬT

4.1. Thiết kế kỹ thuật phần cứng

4.1.1. Sơ đồ đấu dây



Hình 4.1.1 Sơ đồ đấu dây

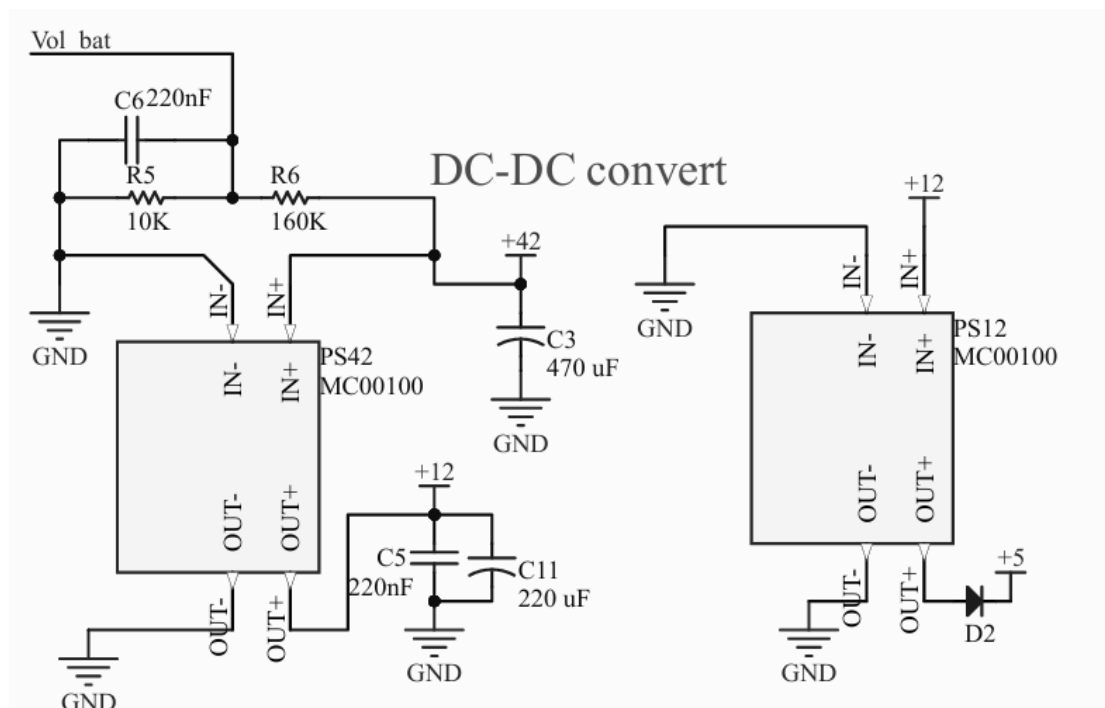
- Cấu hình ngắt ngoài (chế độ CHANGE) tại các chân Pin 36, Pin 39 để đo độ rộng xung tín hiệu (1000-2000 μ s) từ RC Receiver thông qua mạch cầu phân áp.
- Thiết lập ngắt ngoài ưu tiên cao tại các chân Pin 34 (S₁), 35 (S₂), 33 (S₃) cho cụm bánh trái và Pin 25 (S₄), 26 (S₅), 21 (S₆) cho cụm bánh phải để đếm xung vuông từ mạch gia công tín hiệu cảm biến Hall của 6 bánh xe.
- Sử dụng bộ chuyển đổi tương tự-số ADC1 (Kênh CH4) tại chân Pin 32 để đo điện áp Pin nguồn từ mạch cầu phân áp.
- Cấu hình ngoại vi phần cứng LEDC của ESP32 xuất xung PWM ở tần số cố định 5kHz (độ phân giải 8-bit) trên 6 kênh độc lập. Các kênh này được gán tương ứng với Pin 27 (T₁), 14 (T₂), 13 (T₃) cho cụm trái và Pin 17 (T₄), 16 (T₅), 5 (T₆) cho cụm phải.
- Thiết lập chế độ OUTPUT kỹ thuật số cho các chân Pin 23 (u_b1), 18 (u_b2) xuất tín hiệu phanh và Pin 22 (u_d1), 19 (u_d2) xuất tín hiệu đảo chiều qua mạch gia công tín hiệu để kích hoạt chế độ phanh và đảo chiều.

Tài nguyên ESP32	Năng lực ESP32	Sử dụng
Tổng số GPIO khả dụng	25 chân	19 chân (10 chân Output, 9 chân Input)
Bộ định tuyến PWM (LEDC)	16 kênh hoạt động độc lập	6 kênh (Tần số 5 kHz, độ phân giải 8-bit)
Ngắt ngoài (External Interrupt)	Hỗ trợ trên tất cả các GPIO	8 GPIO (6 kênh Hall + 2 kênh tay cảm RC)
Bộ chuyển đổi Analog-Digital	8 Kênh (Thuộc bộ ADC1 an toàn khi có WiFi)	1 Kênh (Nội suy điện áp Pin)
Lõi xử lý (CPU Cores)	2 Lõi (Dual-Core Xtensa LX6 240MHz)	2 Lõi (Phân bổ bởi FreeRTOS)

Bảng 4.1.1 Thống kê tài nguyên ESP32

4.1.2. Mạch cung cấp nguồn

Hệ thống Robot 6x6 sử dụng nguồn năng lượng chính từ bộ Pin 42V. Tuy nhiên, các linh kiện điện tử trên bo mạch trung tâm yêu cầu các mức điện áp hoạt động khác nhau (12V, 5V, 3.3V). Để đảm bảo hệ thống vận hành ổn định, không bị sụt áp khi tải nặng và hạn chế tối đa nhiễu điện từ (EMI), khối nguồn được thiết kế theo cấu trúc hạ áp nhiều cấp kết hợp các mạch bảo vệ chủ động.



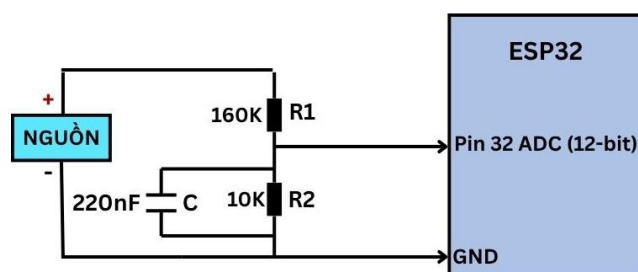
Hình 4.1.2 Sơ đồ nguyên lý khối mạch cung cấp nguồn (DC-DC Convert)

Dựa trên sơ đồ nguyên lý, cấu trúc cấp nguồn được chia thành các phân hệ sau:

- Khối bảo vệ đầu vào: Nguồn 42V từ Pin trước khi đi vào mạch hạ áp được đi qua Cầu chì F1 (3A) để bảo vệ ngắn mạch và quá dòng. Tiếp đó, dòng điện đi qua Diode D1 nhằm mục đích chống phân cực ngược, ngăn chặn hoàn toàn nguy cơ cháy nổ board mạch nếu người dùng cắm ngược cực pin. Cuối cùng, tín hiệu được làm phẳng bởi tụ hóa C3 (470uF) để lọc nhiễu tần số thấp.
- Cấp hạ áp thứ nhất (42V xuống 12V): Sử dụng module hạ áp thứ nhất (IC LM2596HVS) để chuyển đổi điện áp từ 42V xuống 12V. Mức điện áp 12V này đóng vai trò cực kỳ quan trọng, được sử dụng làm nguồn nuôi (Vcc) chuyên dụng cho các bộ khuếch đại thuật toán (Op-Amp LM358) trong mạch DAC. Việc sử dụng nguồn 12V cho Op-Amp giúp dải điện áp ngõ ra có thể đạt mức 5V tuyến tính một cách hoàn hảo mà không bị hiện tượng xén đỉnh. Ngõ ra 12V được lọc nhiễu tần số cao bằng tụ gốm C5 (220nF) và tụ hóa C11 (220uF).
- Cấp hạ áp thứ hai (12V xuống 5V): Thay vì hạ áp trực tiếp từ 42V xuống 5V gây tỏa nhiệt lượng lớn do chênh lệch áp cao, thiết kế sử dụng phương pháp nối tiếp. Module hạ thứ hai (IC LM2596HVS) lấy nguồn đầu vào là 12V từ cấp thứ nhất để hạ xuống 5V. Nguồn 5V này được dùng để cung cấp năng lượng cho mạch thu RC, các cảm biến Hall, IC Logic và cấp vào chân VIN của ESP32. Ngõ ra 5V được chặn bởi Diode D2 để ngăn hiện tượng dội ngược điện áp khi người dùng cắm cáp USB vào ESP32 để nạp code.

4.1.3. Khối thu nhận tín hiệu đầu vào

a.Mạch đo điện áp Pin



Hình 4.1.3 Sơ đồ nguyên lý mạch đo điện áp Pin

Hệ thống Robot 6x6 sử dụng nguồn năng lượng chính là bộ Pin 42V. Để giám sát dung lượng pin và hiển thị lên giao diện WebServer/ESP-NOW nhằm đưa ra cảnh

bảo an toàn khi pin yếu, vì điều khiển ESP32 cần phải đọc được giá trị điện áp này. Tuy nhiên, dải điện áp đầu vào cho phép của bộ chuyển đổi tương tự - số (ADC) trên ESP32 chỉ giới hạn ở mức tối đa 3.3V. Do đó, việc cắm trực tiếp nguồn 42V vào vi điều khiển sẽ gây hư hỏng lập tức. Một mạch gia công tín hiệu hạ áp là bắt buộc.

- Nguyên lý hoạt động và Tính toán linh kiện: Dựa trên sơ đồ nguyên lý (Hình 4.1.3), cấu trúc mạch đo pin bao gồm các thành phần sau:

- + Mạch cầu phân áp: Được thiết kế với hai điện trở $R_1 = 160k\Omega$, $R_2 = 10k\Omega$ mắc nối tiếp. Việc lựa chọn giá trị điện trở hàng trăm kilo-Ohm giúp giảm thiểu tối đa dòng điện rò chạy qua mạch, từ đó tránh gây tổn hao năng lượng của Pin. Theo định luật phân áp, dải điện áp tối đa mà mạch có thể đo lường an toàn (khi điện áp tại chân ADC đạt ngưỡng 3.3V) được tính như sau:

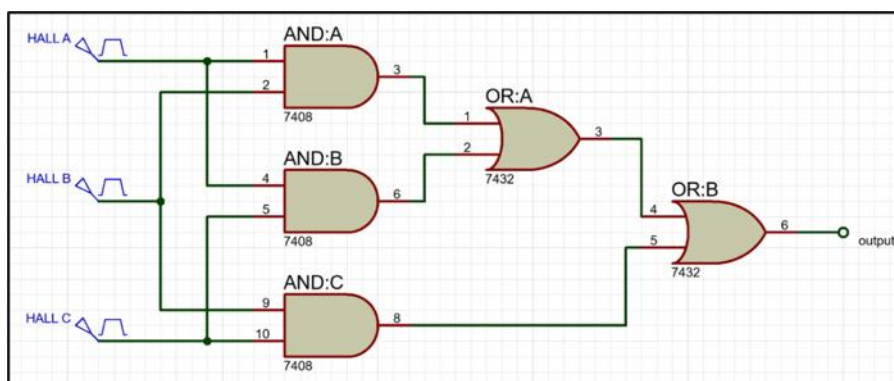
$$V_{max_input} = V_{ADC_max} \times \frac{R_1 + R_2}{R_2} = 3.3 \times \frac{160 + 10}{10} = 56.1 \text{ (V)}$$

Với ngưỡng đo tối đa là 56.1V, mạch hoàn toàn đáp ứng an toàn và có dải dự trữ rộng cho hệ pin 42V.

- + Mạch lọc nhiễu thông thấp: Một tụ điện gốm $C = 220nF$ được mắc song song với điện trở R_2 . Khi 6 động cơ BLDC và các module hạ áp hoạt động sẽ sinh ra các xung nhiễu điện từ (EMI) và nhiễu cao tần trên đường dây nguồn. Tụ điện này đóng vai trò triệt tiêu các gai nhiễu cao tần dẫn xuống Mass (GND), đảm bảo dòng điện áp đưa vào chân vi điều khiển là tín hiệu một chiều (DC) tuyến tính và phẳng nhất có thể.

b.Mạch nhân xung tín hiệu Hall

Mục tiêu chính của Mạch nhân xung là tăng độ phân giải của tín hiệu phản hồi tốc độ từ Motor. Mạch này thu thập tín hiệu từ ba dây cảm biến Hall (A, B, C) và tổng hợp chúng để tạo ra tín hiệu đầu ra có tần số xung cao gấp ba lần tần số xung của một kênh Hall đơn lẻ. Do giải thuật đo tốc độ dựa trên việc đếm số xung N trong một khoảng thời gian T_w cố định, việc nhân xung giúp tăng số lượng xung đếm được lên gấp ba lần. Điều này cải thiện đáng kể độ chính xác của phép đo, đặc biệt quan trọng khi động cơ hoạt động ở dải tốc độ thấp, nơi số lượng xung đếm được trong thời gian T_w thường rất ít và dễ gây ra sai số lớn.

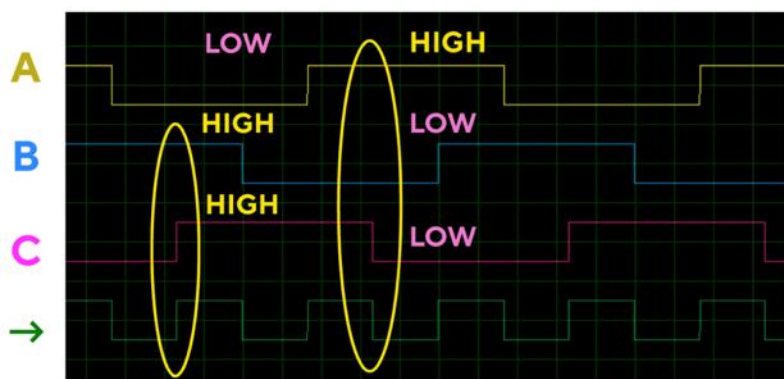


Hình 4.1.4 Sơ đồ nguyên lý mạch nhân xung

Mạch nhân xung này sử dụng các cổng logic AND và OR để tổng hợp các tín hiệu xung từ ba kênh Hall (A, B, C) của Motor. Quá trình tổng hợp được thực hiện theo logic sau:

- Mục tiêu: Đầu ra của Mạch Logic (Output) sẽ là mức cao bất cứ khi nào có ít nhất hai trong ba tín hiệu Hall (A, B, C) ở mức cao.
- Công thức Logic đầu ra cuối cùng: **OUTPUT** = **(A.B) + (A.C) + (B.C)**
- Cấu trúc Cổng Logic:
 - + Cổng AND: Sử dụng để tìm các cặp xung đồng thời ở mức cao (**A.B, A.C, B.C**).
 - + Cổng OR: OR:A tổng hợp xung từ hai khối **AND:A (A.B)** và **AND:B (A.C)**. Công thức: **Z_{OR:A} = (A.B) + (A.C)**.
 - + OR:B (Đầu ra cuối cùng) tổng hợp đầu ra của **OR:A** và khối **AND:C (B.C)**. Công thức: **OUTPUT = Z_{OR:A} + (B.C)**

Như vậy sự kết hợp giữa các cổng Logic AND và OR sẽ đảm bảo mỗi khi 2 trong 3 cảm biến Hall có tín hiệu ở mức HIGH thì đầu ra sẽ là mức HIGH, các trường hợp còn lại thì đầu ra là LOW. Và với nguyên lý hoạt động dựa trên logic này, khi tổng hợp xung từ 3 kênh Hall (xung A, B, C trong hình 4.1.4) thì tín hiệu đầu ra sẽ được nhân tần số xung lên gấp 3 lần (Xung màu xanh lá trong hình 4.1.5) so với việc chỉ sử dụng tín hiệu từ 1 kênh Hall đơn lẻ.



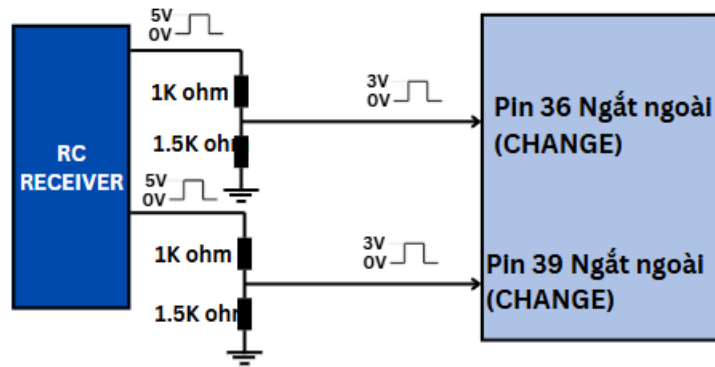
Hình 4.1.5 Kết quả mô phỏng tín hiệu đầu ra (Xung màu xanh lá) của Mạch nhân xung bằng Proteus.

c. Tay cầm RC

Hệ thống sử dụng tay cầm RC (Radio Control) thông qua sóng vô tuyến 2.4GHz làm một trong những nguồn điều khiển chính. Bộ thu tín hiệu (RC Receiver) xuất ra các chuỗi xung điều khiển (PWM/PPM) mang mức điện áp logic là 5V. Trong khi đó, các chân GPIO của vi điều khiển trung tâm ESP32 chỉ hoạt động an toàn ở mức điện áp tối đa là 3.3V. Do đó, việc thiết kế một mạch giao tiếp trung gian để chuyển đổi mức điện áp (Logic Level Converter) là yêu cầu bắt buộc nhằm bảo vệ vi điều khiển khỏi hiện tượng quá áp gây chập cháy phần cứng.



Hình 4.1.6 Tay cầm RC và RC Receiver



Hình 4.1.7 Sơ đồ nguyên lý mạch đọc tín hiệu RC

Dựa trên sơ đồ nguyên lý (Hình 4.1.6), khối mạch đọc tín hiệu RC được thiết kế tối giản nhưng hiệu quả bằng phương pháp sử dụng mạch cầu phân áp điện trở (Voltage Divider). Cụ thể, trên mỗi kênh tín hiệu (đại diện cho ga - Throttle và góc lái - Steering), mạch sử dụng hai điện trở mắc nối tiếp bao gồm $R_1 = 1k\Omega$ và $R_2 = 1.5k\Omega$.

Theo định luật phân áp, mức điện áp tín hiệu cấp vào chân vi điều khiển khi RC Receiver xuất xung ở mức điện áp cao nhất (HIGH = 5V) được tính toán như sau:

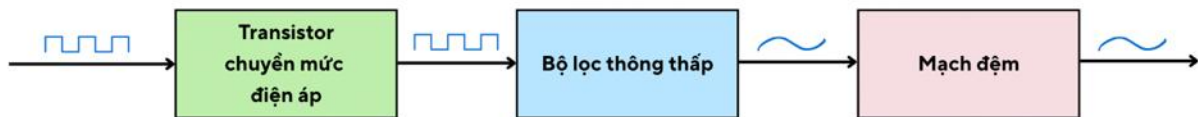
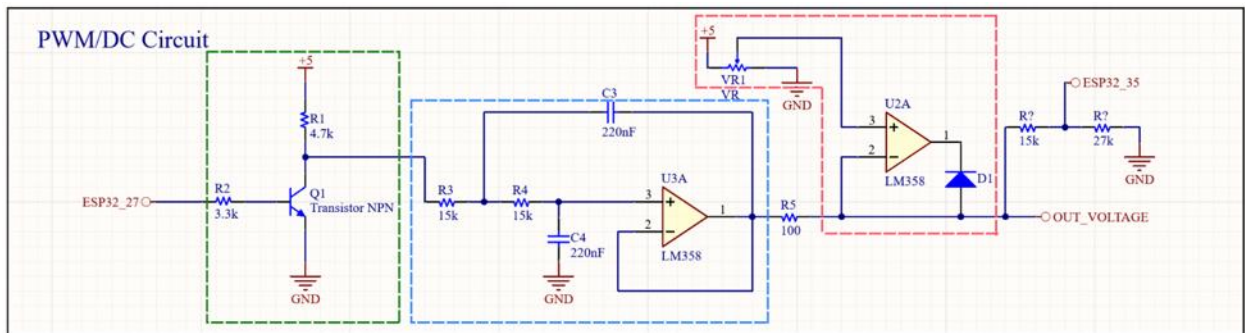
$$V_{out} = V_{in} \times \frac{R_2}{R_1 + R_2} = 5 \times \frac{1.5}{1 + 1.5} = 3 \text{ (V)}$$

Kết quả 3V này nằm hoàn toàn trong ngưỡng mức logic HIGH an toàn tuyệt đối của ESP32 (ngưỡng chuẩn 3.3V), giúp vi điều khiển đọc tín hiệu ổn định mà không bị suy hao hay làm hỏng cổng I/O.

4.1.4. Khối xử lý tín hiệu đầu ra

a. Mạch PWM/DC

Mạch PWM/DC có tác dụng biến tín hiệu xung PWM từ ESP32 thành điện áp tương tự (Analog), tín hiệu điện áp tương tự này sẽ dùng để cấp cho chân tín hiệu ga của Bộ điều khiển động cơ (Motor Drive), từ đó điều khiển được tốc độ Motor.

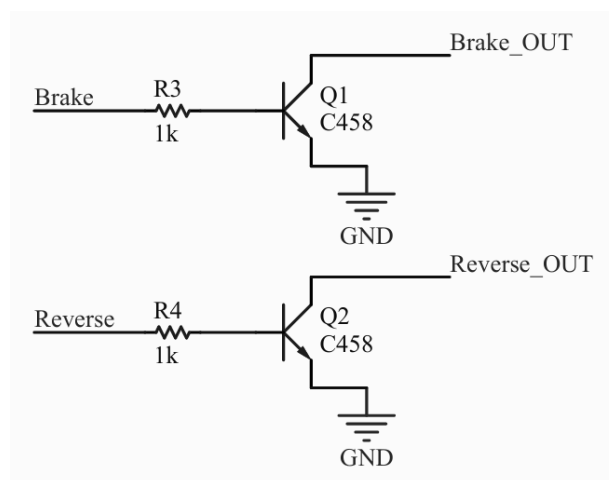


Hình 4.1.8 Sơ đồ nguyên lý mạch PWM/DC

Giải thích về các thành phần có trong mạch PWM/DC:

- Tín hiệu đầu vào (Tín hiệu PWM): Bộ vi điều khiển (ESP32) tạo ra một tín hiệu PWM, tín hiệu này có mức điện áp 3.3V và tần số 50kHz.
- Transistor chuyển mức điện áp: Sau khi qua Transistor, tín hiệu sẽ được nâng biên độ điện áp từ mức **0 – 3.3V** lên **0 – 5V**.
- Bộ lọc thông thấp: Chức năng của bộ lọc này là loại bỏ các tần số cao của tín hiệu PWM và chỉ giữ lại thành phần điện áp trung bình. Quá trình này giúp biến đổi tín hiệu PWM thành một điện áp DC ổn định.
- Mạch đệm: Điện áp DC sau khi được lọc sẽ đi qua mạch đệm. Mạch đệm có tác dụng bảo vệ tín hiệu, đảm bảo điện áp đầu ra không bị ảnh hưởng bởi tải (Motor Driver) và giữ cho tín hiệu ổn định.

b.Mạch đệm điều khiển phanh và đảo chiều



Hình 4.1.9 Sơ đồ nguyên lý mạch đệm điều khiển phanh và lùi

Chức năng đảo chiều của Motor Driver xe điện được kích hoạt khi chân tín hiệu đảo chiều được nối xuống Mass. Để vi điều khiển có thể thực hiện thao tác này an toàn, ta sử dụng một transistor NPN (ví dụ 2n222 hoặc tương đương) mắc theo cấu hình Emitter chung.

- Trạng thái Tiến : Vi điều khiển xuất mức logic 0 (0V) vào cực Base → Transistor ngắt (VCE hở mạch) → Chân đảo chiều của Motor Driver ở trạng thái thả nổi (được kéo lên mức cao bởi điện trở nội của Motor Driver) → Động cơ quay chiều thuận.
- Trạng thái Lùi : Vi điều khiển xuất mức logic 1 (3.3V/5V) vào cực Base → Có dòng kích IB làm Transistor dẫn bão hòa → Cực Collector (C) được nối thông với Emitter (E) xuống Mass → Chân đảo chiều của Motor Driver bị kéo xuống mức thấp → Kích hoạt chế độ lùi.

Việc sử dụng mạch đệm Transistor giúp cách ly về mặt dòng điện, bảo vệ chân GPIO của vi điều khiển khỏi các xung nhiễu từ phía động lực.

4.1.5. Gia công mạch PCB

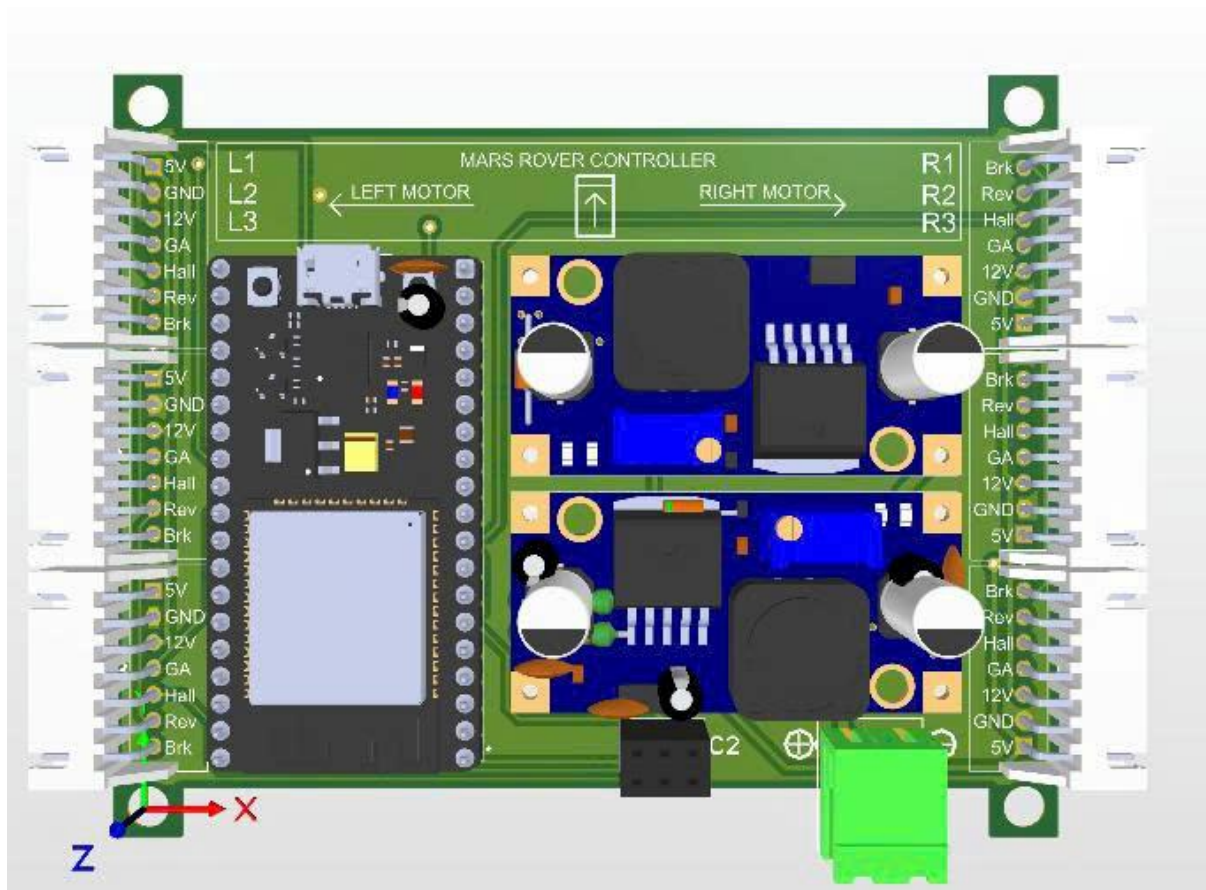
Hệ thống điều khiển của Robot 6x6 bao gồm rất nhiều phân hệ mạch chức năng nhỏ lẻ (mạch nguồn, mạch giao tiếp RC, mạch nhân xung, mạch DAC,...). Nếu sử dụng phương pháp cắm dây (jumpers) hoặc dùng các module rời rạc ghép nối với nhau, hệ thống sẽ gặp phải rủi ro đứt gãy kết nối do rung lắc mạnh khi xe vượt địa hình. Đồng thời, việc đi dây chằng chịt sẽ gây ra hiện tượng nhiễu điện từ (EMI), làm suy hao tín hiệu điều khiển. Do đó, giải pháp thiết kế và gia công các mạch in (PCB - Printed Circuit Board) tích hợp là bước đi tất yếu nhằm tối ưu hóa không gian, nâng cao độ tin cậy và sự ổn định cho toàn bộ hệ thống phần cứng. phần cứng được chia làm 2 cụm bo mạch.

a.Bo mạch Điều khiển Trung tâm

Bo mạch này là trái tim của hệ thống, được thiết kế để tích hợp 3 khối chức năng quan trọng lên cùng một phiên mạch:

- Khối cung cấp nguồn: Tích hợp các IC hạ áp LM2596HVS cùng hệ thống tụ lọc, diode bảo vệ để phân phối các mức điện áp (12V, 5V, 3.3V) từ nguồn Pin 42V đến toàn bộ hệ thống.

- Mạch đo điện áp Pin: Tích hợp trực tiếp cầu phân áp trên board mạch, kết nối thẳng vào chân ADC của ESP32 nhằm rút ngắn tối đa khoảng cách truyền tín hiệu analog, giảm thiểu nhiễu đo lường.
- Mạch giao tiếp Tay cầm RC: Tích hợp mạch chuyển mức điện áp (Logic Level Converter) giúp giao tiếp an toàn giữa tín hiệu 5V của RC Receiver và chân ngắt 3.3V của ESP32.



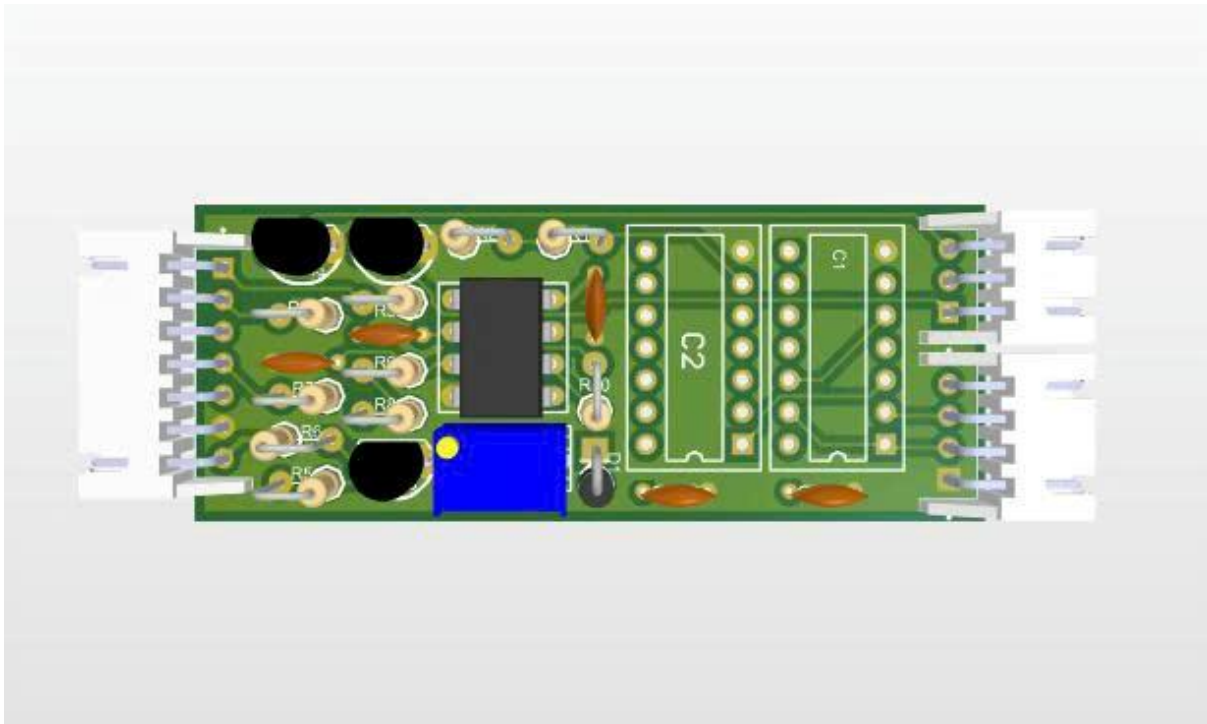
Hình 4.1.10 Mạch cung cấp nguồn + Mạch đo điện áp Pin + Mạch đọc tín hiệu RC

b.Bo mạch Gia công Tín hiệu

Bo mạch thứ hai đóng vai trò là cầu nối giữa Vi điều khiển trung tâm và các Motor Driver công suất lớn. Để điều khiển chính xác các động cơ, bo mạch này tích hợp 3 mạch xử lý chuyên biệt:

- Mạch PWM/DC (DAC): Gồm Transistor chuyển mức, mạch lọc thông thấp (RC filter) và Op-Amp LM358 để biến đổi xung PWM từ vi điều khiển thành điện áp Analog tuyến tính (0-5V).

- Mạch đệm điều khiển phanh và lùi: Tích hợp các Transistor NPN (C458) làm nhiệm vụ đóng cắt tín hiệu logic, bảo vệ cách ly chân GPIO của ESP32 khỏi các dòng điện rò từ Motor Driver.
- Mạch nhân xung Hall: Sử dụng trực tiếp các IC cổng logic (74HC08, 74HC32) hàn trên board để tổng hợp tín hiệu từ 3 pha cảm biến Hall, nhân ba tần số độ phân giải tốc độ truyền về ESP32.

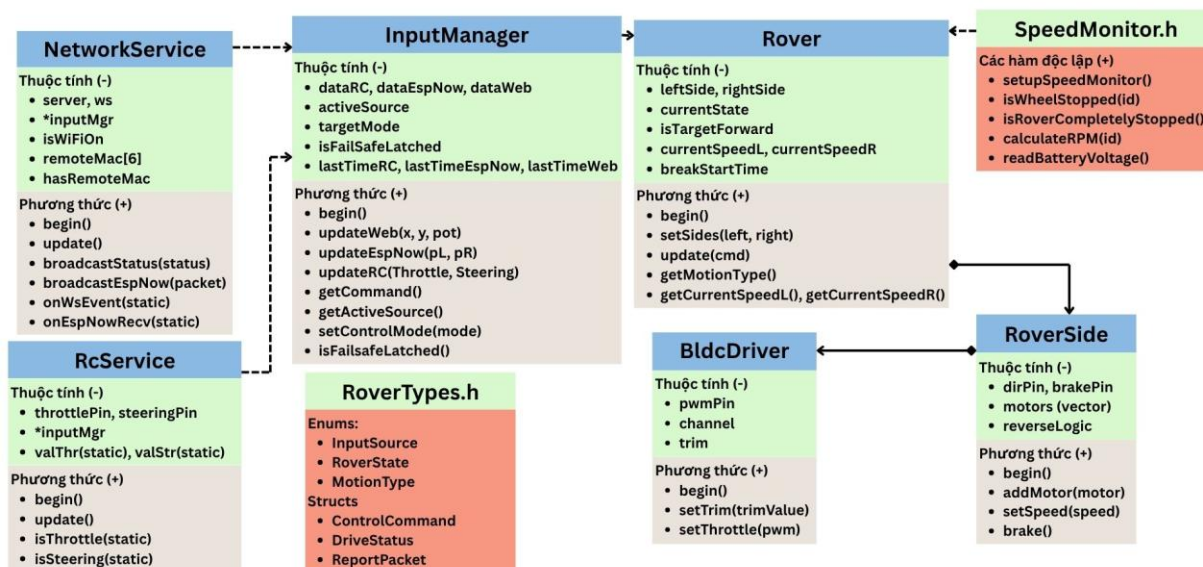


Hình 4.1.11 Mạch PWM/DC + Mạch nhân xung + Mạch đệm điều khiển phanh và lùi

4.2. Thiết kế phần mềm

4.2.1. Kiến trúc phần mềm tổng thể và Mô hình hướng đối tượng (OOP)

Để quản lý hệ thống điều khiển đa tác vụ, điều phối đa luồng truyền thông và vận hành đồng bộ sáu động cơ hành trình, cấu trúc phần mềm lõi được xây dựng hoàn toàn trên nền tảng ngôn ngữ C++ theo phương pháp Lập trình hướng đối tượng (OOP). Việc ứng dụng OOP giúp trừu tượng hóa các khối phần cứng phức tạp thành các thực thể phần mềm độc lập, tăng cường tính đóng gói nhằm bảo vệ các thông số cấu hình và tối ưu hóa khả năng bảo trì hệ thống.



Hình 4.2.1. Sơ đồ cấu trúc lớp OOP

Kiến trúc mã nguồn được phân lớp chặt chẽ và tổ chức thành các lớp đối tượng chức năng sau:

- Lớp BldcDriver (Trừu tượng hóa phần cứng động cơ): Đóng gói toàn bộ thông số vật lý của một liên kết động cơ bao gồm: kênh băm xung LEDC, chân điều khiển phanh kỹ thuật số (Brake) và chân đảo chiều (Reverse). Lớp này cung cấp các phương thức giao tiếp cấp cao như setSpeed(), brake(), setDirection() giúp che giấu cấu trúc thanh ghi vi xử lý bên dưới. Đồng thời, lớp thực hiện thuật toán nghịch đảo phần mềm (255 - Duty) để tương thích với mạch đệm transistor NPN của khối DAC trên PCB công suất.
- Lớp RoverSide (Quản lý cụm truyền động): Đại diện cho một bên phân hệ động lực (cụm ba bánh Trái hoặc cụm ba bánh Phải). Lớp này chứa ba đối tượng BldcDriver, chịu trách nhiệm đồng bộ hóa đồng thời giá trị xung PWM, trạng thái phanh và hướng dịch chuyển cho cả ba động cơ cùng một bên.
- Lớp Rover (Khởi điều phối robot tối cao): Là thực thể phần mềm quản lý trực tiếp hai đối tượng RoverSide (Cụm Trái và Cụm Phải). Lớp này tích hợp cấu trúc Máy trạng thái hữu hạn (FSM) để kiểm soát các mô hình dịch chuyển an toàn và thực thi giải thuật phanh liên khóa Anti-Shock Braking nhằm bảo vệ hộp số cơ khí khi đảo chiều dòng điện đột ngột.

- Lớp InputManager (Quản lý đa hợp nguồn lái): Đóng gói logic kiểm tra cờ phân quyền ưu tiên giữa ba nguồn lệnh điều khiển (Tay cầm RC, WebServer, Tay cầm ESP-NOW). Sau khi chọn lựa được nguồn điều khiển hợp lệ, lớp thực thi giải thuật toán trộn xung hình học lái trượt (Skid-Steer Mixing) để chuyển đổi tọa độ vector Joystick thành giá trị tốc độ mục tiêu có dấu cho hai bên cụm bánh.
- Lớp RcService và NetworkService (Tầng giao tiếp truyền thông): RcService chịu trách nhiệm cấu hình ngắt ngoài bắt sườn xung CHANGE để tính toán chính xác độ rộng xung thô ($1000\mu s - 2000\mu s$) từ bộ thu RC Receiver. NetworkService chịu trách nhiệm khởi tạo mạng WiFi nội bộ (SoftAP Mode), quản lý đường ống truyền tải dữ liệu song công full-duplex chuẩn chuỗi JSON qua WebSocket và thiết lập cấu hình giao thức mạng không dây ngang hàng ESP-NOW.
- Lớp SpeedMonitor (Tầng đo lường hồi tiếp): Đóng gói bộ khóa găng phần cứng portENTER_CRITICAL_ISR() để bảo vệ tính nguyên tử cho các biến đếm ngắt từ 6 cảm biến Hall, tránh tranh chấp tài nguyên đa nhân. Lớp thực thi thuật toán tính toán tốc độ vòng quay (RPM) theo phương pháp đo thời gian M/T tại chu kỳ lấy mẫu 350ms.

4.2.2. Triển khai Hệ điều hành thời gian thực (FreeRTOS)

Hệ thống điều khiển Robot 6x6 yêu cầu giải quyết đồng thời hai bài toán mang tính đối nghịch về mặt thời gian: xử lý luồng dữ liệu mạng nặng nề (WiFi, chuỗi JSON) nhưng vẫn phải đảm bảo tốc độ phản hồi điều khiển cơ khí ở ngưỡng mili-giây. Để giải quyết triệt để vấn đề, chương trình loại bỏ hoàn toàn cấu trúc vòng lặp siêu hạn (Super-loop) truyền thống, thay vào đó ứng dụng Hệ điều hành thời gian thực (FreeRTOS) thông qua các chiến lược sau:

a. Chiến lược phân bổ đa nhân

Tận dụng kiến trúc lõi kép của vi điều khiển ESP32, không gian hoạt động của hệ thống được cô lập thành hai phân khu độc lập thông qua API `xTaskCreatePinnedToCore()`:

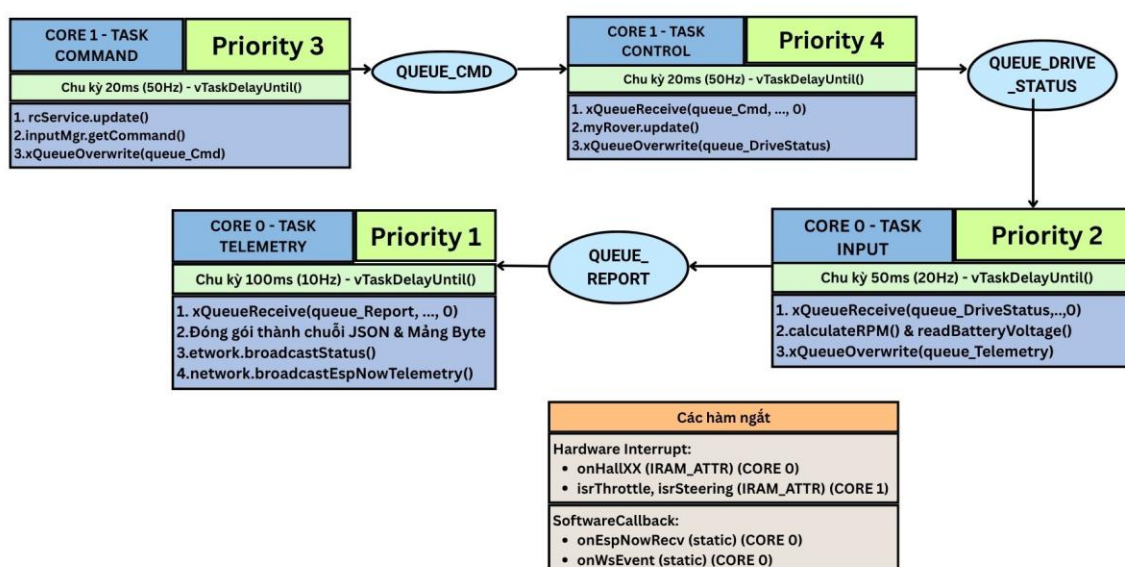
- Core 0 (Xử lý Mạng & Đo lường): Đảm nhiệm các tác vụ đòi hỏi khối lượng tính toán lớn nhưng không nhạy cảm về deadline thời gian thực. Cụ thể là khởi tạo module Radio tần số 2.4GHz, xử lý giải mã/đóng gói các chuỗi dữ liệu JSON dài

và thực hiện phép toán nội suy trung bình (Floating-point Math) cho thông số RPM. Việc ghim các tác vụ này ở Core 0 đảm bảo quá trình truyền thông không bao giờ làm đình trệ hệ thống điều khiển.

- Core 1 (Xử lý Cơ khí & Động học): Core này hoàn toàn không bị ảnh hưởng bởi độ trễ mạng, chỉ tập trung giải quyết các thuật toán đa hợp tín hiệu, tính toán máy trạng thái FSM và trực tiếp băm xung PWM xuống các vi mạch công suất.

b. Thiết kế Tác vụ (Task) và Cơ chế Khóa cứng chu kỳ (vTaskDelayUntil)

Hệ thống được chia nhỏ thành 4 Tác vụ (Task) độc lập, hoạt động theo cơ chế lập lịch ưu tiên độc chiếm của FreeRTOS:



Hình 4.2.2 Sơ đồ cấu trúc các Task FreeRTOS

- Task_Control (Ưu tiên 4): Chạy tại Core 1. Chịu trách nhiệm thực thi Máy trạng thái (FSM) và xuất PWM.
- Task_Command (Ưu tiên 3): Chạy tại Core 1. Gom tín hiệu từ RC, Web, ESP-NOW và phân quyền ưu tiên.
- Task_Input (Ưu tiên 2): Chạy tại Core 0. Lấy dữ liệu đếm xung Hall và ADC điện áp Pin để tính toán thông số.
- Task_Report (Ưu tiên 1): Chạy tại Core 0. Đóng gói dữ liệu thành JSON và phát sóng.

Cơ chế khóa cứng chu kỳ 20ms: Điểm nhấn kỹ thuật của kiến trúc RTOS nằm ở việc loại bỏ lệnh `vTaskDelay()` thông thường. Đối với `Task_Control` và `Task_Command`, chương trình sử dụng API chuyên dụng `vTaskDelayUntil(&xLastWakeTime, 20ms)`. Điểm khác biệt của hàm này là khả năng tự động đo lường thời gian phần mềm đã thực thi lệnh, từ đó bù trừ thời gian ngủ còn lại. Nhờ đó, bất kể thuật toán FSM tính toán nhanh hay chậm, thời điểm bắt đầu chu kỳ mới luôn được khóa chặt chính xác tại mốc 20.00ms (50Hz), triệt tiêu hoàn toàn hiện tượng trôi thời gian.

c. Giao tiếp liên tác vụ (Inter-Task) và Chống tranh chấp vùng nhớ

Khi các tác vụ vận hành song song trên hai lõi CPU, việc chia sẻ biến toàn cục (Global Variable) sẽ lập tức gây ra hiện tượng đụng độ bộ nhớ. Chương trình triển khai hai lớp bảo vệ dữ liệu:

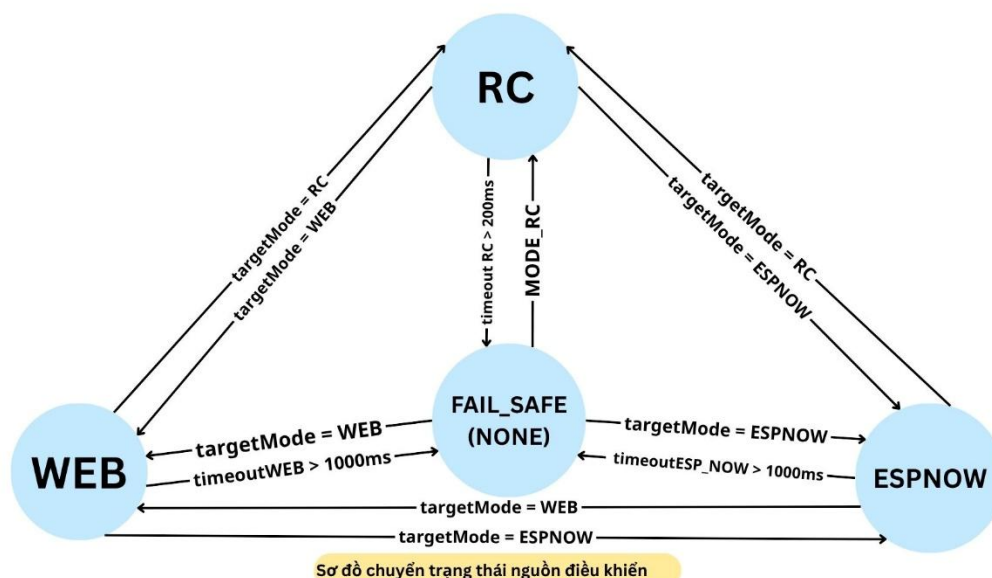
- Sử dụng Hàng đợi (Queue): Dữ liệu truyền từ `Task_Command` sang `Task_Control` được vận chuyển qua các `xQueue`. Cấu trúc này tích hợp sẵn cơ chế Thread-Safe của RTOS, đảm bảo gói dữ liệu (Struct) mệnh lệnh luôn được truyền đi toàn vẹn mà không bị "xé rách" giữa chừng.
- Khóa găng phần cứng (Spinlock): Đối với biến đếm vòng quay từ cảm biến Hall, hệ thống phải đối mặt với áp lực ngắt phần cứng cực kỳ lớn. Ở tốc độ cao, 6 bánh xe có thể tạo ra hàng ngàn ngắt mỗi giây (tại Core 1), trong khi `Task_Input` (tại Core 0) liên tục lôi biến đếm ra để tính RPM. Để chống thất thoát dữ liệu, chương trình triển khai hàm `portENTER_CRITICAL_ISR()`. Cơ chế khóa Spinlock này hoạt động như một ổ khóa toàn cục, tạm thời đóng băng bộ lập lịch của cả Core 0 và Core 1 trong vài nano-giây, giúp dữ liệu đếm xung được cập nhật chính xác tuyệt đối mà không làm vi phạm chu kỳ điều khiển 20ms của hệ thống.

4.2.3. Khối thu nhận và Phân quyền điều khiển

Khối `InputManager` đóng vai trò là trung tâm đa hợp tín hiệu (Multiplexer), chịu trách nhiệm điều phối luồng dữ liệu đầu vào từ ba nguồn khác nhau (RC, WebServer, ESP-NOW), xử lý các tình huống mất kết nối và tính toán nội suy tốc độ cho từng cụm bánh xe.

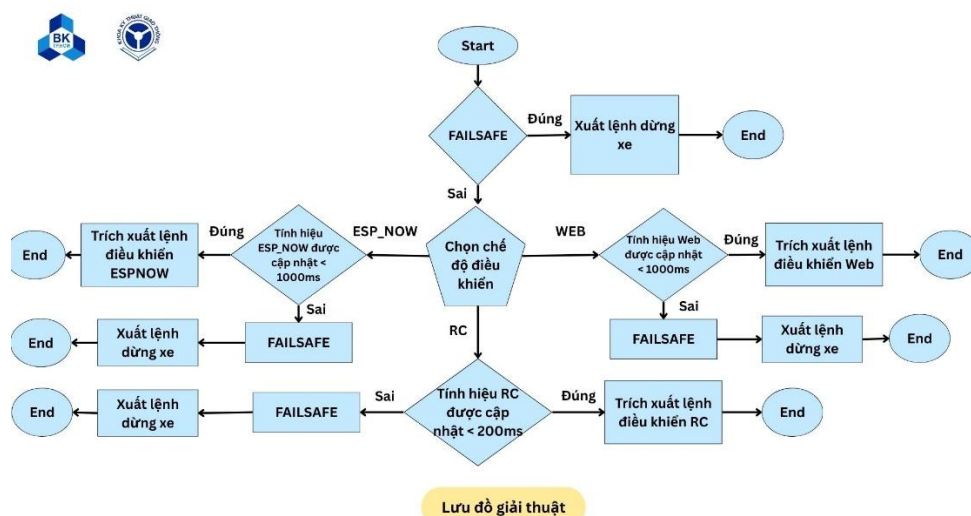
a. Logic đa hợp tín hiệu và Cơ chế bảo vệ an toàn (Timeout Failsafe)

Để đảm bảo an toàn tuyệt đối khi robot vận hành ở tốc độ cao hoặc trong môi trường nhiễu sóng, chương trình được thiết kế một Máy trạng thái hữu hạn (FSM) chuyên biệt để quản lý nguồn điều khiển.



Hình 4.2.3 Sơ đồ chuyển trạng thái nguồn điều khiển

Dựa trên sơ đồ chuyển trạng thái (Hình 4.2.3), hệ thống tồn tại 4 trạng thái điều khiển: WEB, ESPNOW, RC và FAIL_SAFE. Việc chuyển đổi giữa các trạng thái (ví dụ: targetMode = RC) được kích hoạt thông qua lệnh chọn mode từ người dùng trên giao diện giám sát. Tuy nhiên, mọi trạng thái đều có thể bị ép rơi vào trạng thái an toàn FAIL_SAFE nếu xảy ra sự cố gián đoạn truyền thông (Timeout).



Hình 4.2.4 Lưu đồ giải thuật phân quyền và xử lý mất kết nối

Cụ thể, luồng thuật toán được thực thi tuần tự trong Task_Command theo lưu đồ (Hình 4.2.4) như sau:

- Kiểm tra chốt an toàn (Failsafe Latch): Ngay khi bắt đầu chu kỳ 20ms, hệ thống kiểm tra cờ FAILSAFE. Nếu cờ này đang bật (True), thuật toán lập tức bỏ qua mọi tín hiệu đầu vào, xuất lệnh dừng xe khẩn cấp và kết thúc chu kỳ. Điều này ngăn chặn việc xe tự động vọt đi khi có sóng lại mà người dùng chưa sẵn sàng kiểm soát.
- Kiểm tra Timeout theo đặc tính giao thức: Nếu hệ thống an toàn, khối Đa hợp (Multiplexer) sẽ trở tới hàm đọc dữ liệu của nguồn đang được kích hoạt. Tại đây, thuật toán tính toán độ trễ (Delta Time) kể từ lần cuối cùng nhận được gói tin hợp lệ:
 - + Đối với Tay cầm RC: Do tính chất sóng vô tuyến truyền liên tục với tần số cao (50Hz), ngưỡng Timeout được siết chặt ở mức cực thấp là 200ms.
 - + Đối với WebServer và ESP-NOW: Do giao thức mạng có thể xảy ra hiện tượng rớt gói tin (Packet Loss) hoặc độ trễ mạng (Ping/Latency), ngưỡng Timeout được nới lỏng ở mức 1000ms để tránh hệ thống bị phanh gấp liên tục (False positive).
- Thực thi hoặc Cảnh báo: Nếu tín hiệu được cập nhật kịp thời trong ngưỡng an toàn, tọa độ Joystick sẽ được trích xuất. Ngược lại, nếu vượt quá thời gian Timeout, hệ thống lập tức bật cờ FAILSAFE và xuất lệnh khóa phanh dừng xe.

b. Thuật toán trộn xung hình học lái trượt

Đặc thù của Robot 6x6 là không sử dụng cơ cấu thước lái cơ khí (như hệ thống lái Ackermann trên ô tô thông thường) mà rẽ hướng bằng cơ chế Lái trượt (Skid-Steering). Nghĩa là, để xe rẽ trái hoặc phải, hệ thống phải tạo ra sự chênh lệch vận tốc (Differential Speed) giữa cụm 3 bánh bên Trái và cụm 3 bánh bên Phải.

Sau khi tín hiệu từ các bộ điều khiển được InputManager phê duyệt an toàn, chúng sẽ tồn tại ở dạng tọa độ vector 2D gồm hai trục: Trục Y (Throttle - Mức ga Tiến/Lùi) và Trục X (Steering - Góc lái Trái/Phải). Chương trình áp dụng thuật toán trộn xung (Mixing Algorithm) để nội suy ra vận tốc mục tiêu cho từng bên bánh xe theo phương trình động học sau:

$$Speed_{Left} = Y + X, Speed_{Right} = Y - X$$

Xử lý các kịch bản chuyển động:

- Chạy thẳng (Tiến/Lùi): Trục $X = 0$, khi đó $Speed_{Left} = Speed_{Right} = Y$. Hai cụm bánh quay đồng tốc, xe đi thẳng.
- Ôm cua mềm (Soft Turn): Ví dụ người dùng đẩy ga Tiến ($Y > 0$) và rẽ Trái ($X < 0$). Khi đó $Speed_{Right}$ sẽ lớn hơn $Speed_{Left}$. Cụm bánh phải quay nhanh hơn cụm bánh trái, tạo mô-men xoay đẩy mũi xe rẽ sang trái.
- Xoay vòng tại chỗ (Zero-Turn / Pivot): Khi người dùng đứng yên ($Y = 0$) nhưng bẻ lái gắt ($X \neq 0$). Phương trình trở thành: $Speed_{Left} = X$ và $Speed_{Right} = -X$. Hai bên bánh xe sẽ quay ngược chiều nhau với cùng một tốc độ, giúp xe lùi một bên và tiến một bên, tạo ra khả năng xoay compa tại chỗ với bán kính vòng quay bằng 0 (Zero-Radius), cực kỳ hữu dụng trong địa hình hẹp.

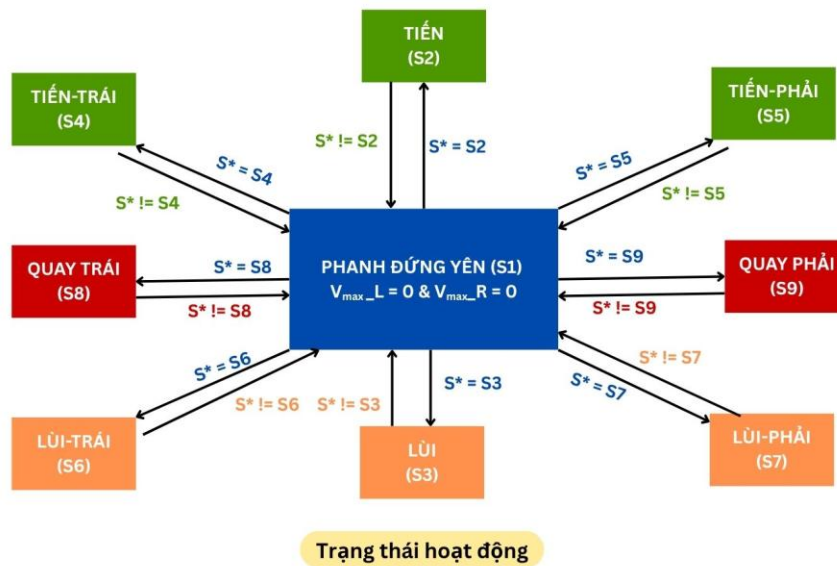
Sau khi tính toán, giá trị $Speed_{Left}$ và $Speed_{Right}$ sẽ được đưa qua hàm giới hạn (Constrain) để đảm bảo không vượt quá dải băm xung PWM cho phép ($0 \rightarrow 255$) trước khi đẩy vào Hàng đợi (Queue) truyền cho Máy trạng thái FSM.

4.2.4. Khối xử lý Động lực học và Máy trạng thái

Sau khi InputManager nội suy ra vận tốc mục tiêu của hai cụm bánh ($Speed_{Left}$ và $Speed_{Right}$), nếu vi điều khiển xuất trực tiếp giá trị này ra các Motor Driver sẽ lập tức gây ra hiện tượng giật cục, sốc dòng điện hoặc nghiêm trọng hơn là mẻ bánh răng hộp số khi xe đột ngột đảo chiều. Để giải quyết triệt để bài toán này, đối tượng Rover được thiết kế tích hợp một Máy trạng thái hữu hạn (FSM) và bộ lọc khởi động mềm.

a. Phân loại trạng thái chuyển động

Dựa vào tổ hợp dấu (âm/dương) và độ lớn của $Speed_{Left}$ và $Speed_{Right}$, Máy trạng thái FSM phân loại hoạt động của Robot thành 9 mô hình chuyển động cốt lõi:



Hình 4.2.5 Sơ đồ trạng thái hoạt động

- Phanh đứng yên: SpeedLeft == 0 và SpeedRight == 0. Hệ thống xả phanh, xe trôi tự do hoặc nhả ga.
- Tiến / Lùi: Cả hai cụm bánh có cùng giá trị và cùng dấu.
- Tiến trái / Tiến phải / Lùi trái / Lùi phải: Cả hai cụm bánh có cùng dấu dương, nhưng chênh lệch về độ lớn.
- Quay trái / Quay phải: Hai cụm bánh có giá trị vận tốc bằng nhau nhưng trái dấu hoàn toàn.

Việc định nghĩa rõ ràng các trạng thái này giúp hệ thống xác định chính xác thời điểm nào cần kích hoạt chân logic Đảo chiều (Reverse) ở từng cụm bánh tương ứng.

b. Giải thuật phanh và đảo chiều:

Bài toán nguy hiểm nhất trong điều khiển xe điện là khi xe đang lao nhanh về phía trước nhưng người dùng đột ngột gạt cần Joystick hết cỡ về phía sau. Nếu phần mềm đảo trạng thái chân Reverse ngay lập tức, dòng điện ngược sinh ra cực lớn sẽ nung chín vi mạch công suất và động năng quán tính sẽ bẻ gãy trục cơ khí.

Để bảo vệ hệ thống, FSM triển khai thuật toán Phanh liên khóa theo chu trình sau:

- Phát hiện xung đột chiều: FSM liên tục so sánh mô hình chuyển động hiện tại (currentState) với mô hình chuyển động mục tiêu (targetState). Nếu phát hiện sự

chuyển đổi mang tính đối nghịch (Ví dụ: Từ nhóm FORWARD sang nhóm BACKWARD), FSM lập tức từ chối lệnh đảo chiều.

- Kích hoạt trạng thái phanh trung gian (MOTION_BRAKING): Hệ thống chặn lệnh xuất PWM, đồng thời xuất mức logic HIGH ra các chân Brake_L và Brake_R để kích hoạt phanh điện từ trên Motor Driver.
- Giám sát vận tốc thời gian thực: Trong lúc phanh, FSM liên tục đọc giá trị RPM thực tế phản hồi từ SpeedMonitor. Chỉ khi nào thuật toán xác nhận vận tốc của toàn bộ 6 bánh xe đã giảm về ngưỡng an toàn ($\text{RPM} \approx 0$) duy trì liên tục trong một khoảng thời gian trễ nhất định (Brake Hold Time), FSM mới "mở khóa".
- Cho phép đảo chiều: Sau khi xe đã dừng hẳn, FSM mới cho phép thay đổi trạng thái chân Reverse và bắt đầu xuất xung PWM để xe chạy theo hướng mới.

c. Bộ lọc khởi động mềm (Soft-start Ramp Step)

Bên cạnh thuật toán chống sóc hướng, hệ thống còn được trang bị thuật toán chống sóc tốc độ. Thay vì gán ngay lập tức giá trị PWM hiện tại bằng với PWM mục tiêu (gây ra dòng khởi động Inrush Current rất lớn), chương trình áp dụng hàm nội suy bậc thang (Ramp Step).

Theo đó, trong mỗi chu kỳ 20ms của Task_Control, giá trị PWM thực tế xuất ra động cơ chỉ được phép tăng hoặc giảm một lượng giới hạn (Step Size) cho đến khi bắt kịp giá trị mục tiêu. Thuật toán này đóng vai trò như một bộ phụộc nhún phần mềm, giúp Robot tăng tốc và giảm tốc cực kỳ êm ái, bám đường tốt và tối ưu hóa tuổi thọ của bộ Pin.

4.2.5. Khối Đo lường và Giám sát

Do Task_Input đảm nhiệm trên Core 0, đóng vai trò thu thập các tín hiệu hồi tiếp từ môi trường vật lý, xử lý lọc nhiễu bằng thuật toán phần mềm và tính toán ra các thông số vận hành cuối cùng để phục vụ cho việc hiển thị và giám sát.

a. Thuật toán phần mềm đo lường vận tốc (M/T Method)

Để tính toán chính xác tốc độ vòng quay (RPM) của 6 bánh xe, hệ thống áp dụng phương pháp M/T (kết hợp đếm số xung và thời gian) với chu kỳ lấy mẫu cố định là 350ms. Tuy nhiên, để đảm bảo an toàn vùng nhớ (Thread-safe) trong môi trường RTOS đa nhân, thuật toán phần mềm được triển khai chia làm hai bước phân lập:

- Thu thập dữ liệu thô (Tại Core 1): Hàm phục vụ ngắt (ISR) gắn trên các chân cảm biến Hall được tối ưu hóa ở mức tối đa bằng cách lưu vào RAM nội (IRAM_ATTR). Mỗi khi có sườn xung xuất hiện, ISR chỉ thực hiện duy nhất hai thao tác là tăng biến đếm (count++) và lưu lại tem thời gian của xung cuối cùng (last_tick). Việc không chứa bất kỳ phép tính toán nào trong ISR giúp giảm tải tuyệt đối cho Core 1.
- Xử lý số liệu (Tại Core 0): Định kỳ mỗi 50ms, Task_Input sẽ gọi hàm tính toán RPM. Tại đây, chương trình sử dụng khóa găng phần cứng portENTER_CRITICAL(&timerMux) để đóng băng ISR trong vài nano-giây, tiến hành sao chép an toàn biến count và last_tick ra các biến cục bộ, sau đó reset bộ đếm về 0 và lập tức mở khóa. Toàn bộ các phép tính toán số học dấu phẩy động (Floating-point Math) phức tạp theo công thức M/T đều được thực thi ở môi trường ngoài ngắt trên Core 0. Giải pháp này vừa triệt tiêu được rủi ro "xé dữ liệu", vừa không làm đình trệ hệ thống điều khiển cơ khí.

b. Thuật toán đo lường và lọc nhiễu điện áp Pin

Tín hiệu điện áp analog tuyến tính từ mạch cầu phân áp được đo thông qua bộ chuyển đổi ADC1 (độ phân giải 12-bit) của ESP32 tại chân IO32.

Tuy nhiên, do đặc thù môi trường hoạt động chứa các tải cảm kháng lớn (6 động cơ BLDC và các mạch hạ áp Switching), tín hiệu analog thu về thường xuyên xuất hiện các gai nhiễu hoặc bị sụt áp ảo tức thời khi động cơ khởi động. Để khắc phục, phần mềm không sử dụng trực tiếp giá trị lấy mẫu đơn lẻ mà tích hợp một bộ lọc số học (Digital Filter):

- Bộ lọc lấy mẫu trung bình: Phần mềm tiến hành lấy mẫu liên tục tín hiệu ADC nhiều lần trong một chu kỳ, sau đó loại bỏ các giá trị nhiễu đột biến và tính trung bình cộng.
- Giá trị trung bình này sau đó mới được đưa vào công thức nhân với hệ số phân áp (Tỷ lệ điện trở $160k\Omega$ và $10k\Omega$) để nội suy ra giá trị điện áp Pin 42V thực tế. Kỹ thuật lọc phần mềm này kết hợp với tụ lọc thông thấp trên phần cứng tạo ra một thông số điện áp cực kỳ mượt mà, giúp cảnh báo Pin trên giao diện Web chính xác và không bị dao động ảo.

4.2.6. Khối Truyền thông và Giao diện

Khối truyền thông mạng được ghim hoàn toàn trên Core 0, đảm nhiệm vai trò cầu nối thông tin giữa Vi điều khiển trung tâm và người dùng thông qua hai giao thức mạng chạy song song.

a. Mạng không dây nội bộ (SoftAP) và Giao thức WebSocket

Để cho phép các thiết bị di động (Smartphones, Laptop) điều khiển Robot mà không cần thông qua mạng Internet hay Router trung gian, ESP32 được cấu hình hoạt động ở chế độ Điểm truy cập mềm (SoftAP). Nó tự động phát ra một mạng WiFi nội bộ, đồng thời khởi tạo một Máy chủ Web (WebServer).

- Giao thức WebSocket song công: Khác với HTTP truyền thông, chương trình tích hợp giao thức WebSocket để tạo ra một "đường ống" truyền tải dữ liệu liên tục (Full-duplex). Khi người dùng vuốt Joystick trên Web, lệnh điều khiển được gửi xuống ESP32 ngay lập tức; đồng thời chiều ngược lại, ESP32 liên tục gửi thông số vận tốc, điện áp Pin lên Web mà không cần tải lại (reload) trang.
- Đóng gói dữ liệu JSON: Để đảm bảo tính toàn vẹn và dễ dàng bóc tách dữ liệu, toàn bộ thông số giám sát (PWM, RPM 6 bánh xe, Điện áp Pin, Chế độ hoạt động, Trạng thái kết nối) được đối tượng NetworkService đóng gói thành một chuỗi định dạng chuẩn JSON duy nhất trước khi phát sóng định kỳ mỗi 100ms.

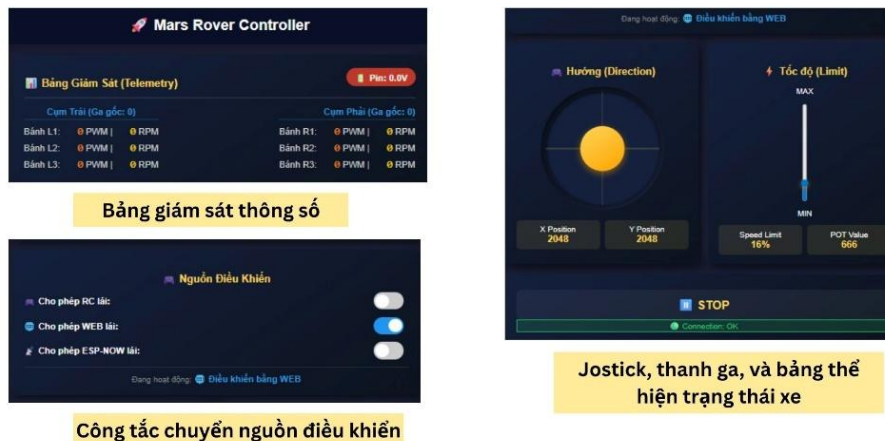
b. Giao thức truyền thông ngang hàng ESP-NOW

Bên cạnh WebServer, hệ thống tích hợp giao thức ESP-NOW chuyên dụng cho Tay cầm điều khiển vật lý. Đây là giao thức truyền tải gói tin (Struct Packet) trực tiếp qua địa chỉ MAC vật lý của các chip ESP32 mà không cần kết nối WiFi tiêu chuẩn.

Nhờ loại bỏ các thủ tục bắt tay (Handshake) rườm rà, ESP-NOW mang lại độ trễ cực thấp (chỉ vài mili-giây), rất lý tưởng cho việc điều khiển xe địa hình ở tốc độ cao, đồng thời cho phép truyền ngược dữ liệu Telemetry về màn hình LCD trên tay cầm.

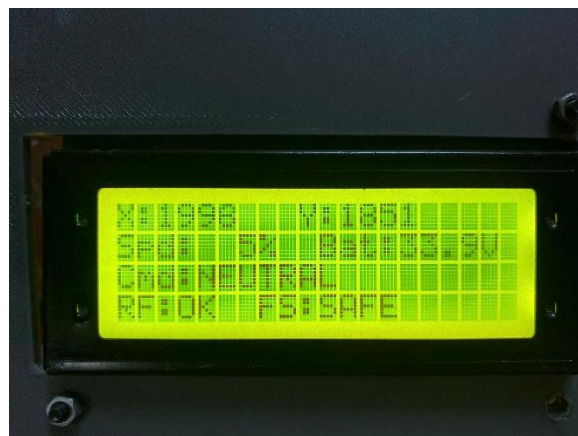
c. Thiết kế Giao diện Giám sát (UI)

Giao diện người dùng được thiết kế trực quan hóa theo thời gian thực:



Hình 4.2.6. Giao diện Web

- Giao diện WebServer: Được lập trình bằng HTML/CSS/JS, tích hợp Joystick ảo 2D để nội suy vector đa hướng, thanh trượt giới hạn tốc độ (Speed Limit), công tắc chuyển đổi nguồn điều khiển và một Bảng Dashboard hiển thị song song PWM/RPM của cụm bánh Trái và Phải, kèm theo đồ họa cảnh báo mức Pin.



Hình 4.2.7. Giao diện Tay cầm ESP-NOW

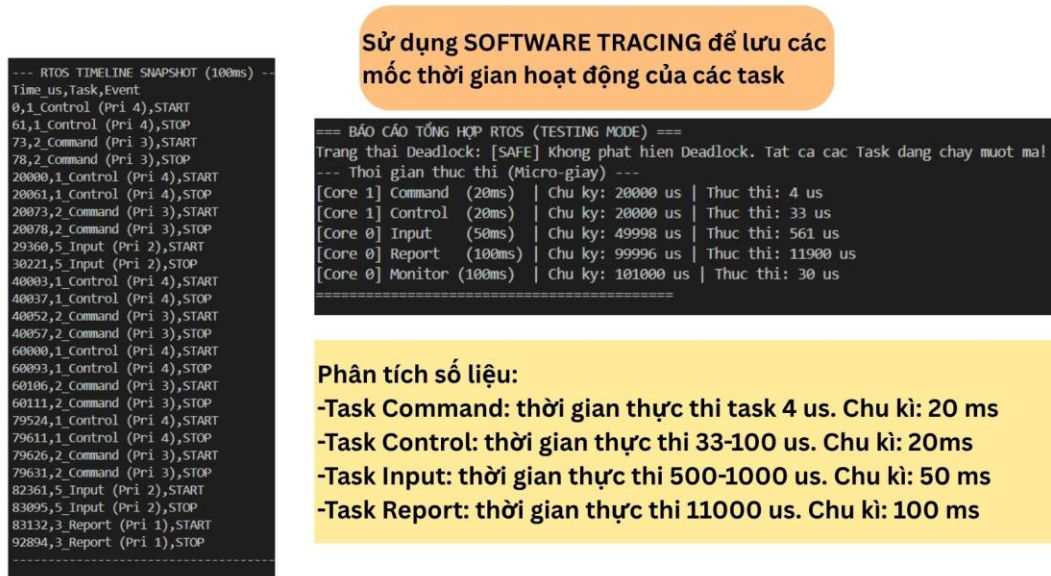
- Giao diện Tay cầm ESP-NOW: Sử dụng màn hình LCD 20x4, hiển thị dạng văn bản tinh gọn các thông số tọa độ trục (X, Y), cấp độ tốc độ, trạng thái máy FSM, điện áp Pin và tình trạng kết nối sóng.

4.3. Kiểm tra thiết kế kĩ thuật

4.3.1. Đánh giá Hiệu năng Hệ thống

Để chứng minh tính đúng đắn của việc ứng dụng hệ điều hành thời gian thực (FreeRTOS) và kiến trúc lõi kép, hệ thống sử dụng phương pháp đo lường phần mềm

(Software Tracing) bằng cách ghi nhận tem thời gian (micros()) trước và sau khi thực thi các Tác vụ.



Hình 4.3.1 Kết quả đo kiểm tra thời gian hoạt động tác vụ

Kết quả phân tích cho thấy:

- Tại Core 0 (Luồng mạng): Task_Report và Task_Input mất từ $500\mu s$ đến $12.000\mu s$ để hoàn thành việc định tuyến WiFi và đóng gói chuỗi JSON. Nếu không dùng RTOS đa nhân, khối lượng công việc khổng lồ này chắc chắn sẽ làm nghẽn toàn bộ hệ thống cơ khí.
- Tại Core 1 (Luồng cơ khí): Nhờ được cách ly hoàn toàn, Task_Command chỉ tốn khoảng $6\mu s$ để phân quyền, và Task_Control tốn chưa đến $100\mu s$ để chạy xong toàn bộ thuật toán FSM và xuất 6 kênh PWM.
- Kết luận: Tổng thời gian CPU Core 1 bận rộn chỉ chiếm khoảng 0.5% trong quỹ thời gian của một chu kỳ $20ms$ ($20.000\mu s$). Hệ thống có dư dả 99.5% thời gian nhàn rỗi để xử lý hàng ngàn ngắt phần cứng từ 6 bánh xe mà không bao giờ xảy ra hiện tượng "cháy deadline" hay trễ chu kỳ điều khiển. Điều này khẳng định hệ thống đạt chuẩn Thời gian thực (Hard Real-Time) một cách xuất sắc.

4.3.2. Kiểm tra chức năng giám sát điện áp Pin

Để đánh giá độ chính xác của phân hệ đo lường năng lượng, tiến hành thực nghiệm đo đặc điện áp Pin song song bằng hai phương pháp: đọc dữ liệu trả về trên

Bảng giám sát (Telemetry) của giao diện WebServer và đo trực tiếp tại hai cực của Pin bằng đồng hồ vạn năng số (VOM) chuẩn.



Hình 4.3.2 Đối chiếu điện áp Pin trên giao diện Web và đồng hồ VOM)

Kết quả thực nghiệm:

- Chức năng truyền tải: Hệ thống đã thu thập và hiển thị thành công giá trị điện áp Pin lên giao diện Web theo thời gian thực. Tín hiệu được cập nhật mượt mà, không bị dao động ảo (nhờ thuật toán bộ lọc trung bình Moving Average trên phần mềm và tụ lọc 220nF trên phần cứng).
- Đánh giá sai số: Khi đối chiếu tại cùng một thời điểm, thiết bị đo tiêu chuẩn (VOM) ghi nhận điện áp thực tế là 38.8V, trong khi dữ liệu nội suy trên giao diện Web hiển thị 34.8V. Sai số ghi nhận vào khoảng 4.0V (tương đương khoảng 10%).

Phân tích nguyên nhân và giải pháp:

- Dung sai linh kiện điện trở: Mạch cầu phân áp sử dụng điện trở 160kΩ và 10kΩ. Với sai số linh kiện thông thường (1% - 5%), khi nhân với mức điện áp cao (hơn 30V), độ lệch điện áp đưa vào chân vi điều khiển sẽ bị khuếch đại lên đáng kể.
- Đặc tính không tuyến tính của ADC ESP32: Bộ ADC của dòng chip ESP32 được biết đến với đặc điểm không tuyến tính ở các vùng điện áp sát ngưỡng 0V và 3.3V, dẫn đến kết quả nội suy phần mềm bị lệch so với thực tế.
- Giải pháp khắc phục: Hệ thống hoàn toàn đáp ứng được tính năng cảnh báo (khi pin sụt áp dần đều). Để đạt độ chính xác tuyệt đối trong các phiên bản thương

mại hóa, cần sử dụng điện trở chính xác cao (sai số 0.1%) và bổ sung một dải hệ số bù trừ vào mã nguồn phần mềm.

4.3.3. Kiểm tra chức năng giám sát tốc độ động cơ (RPM)

Chức năng giám sát tốc độ (RPM) là một bài toán khó do vi điều khiển phải đồng thời đếm xung Hall tốc độ cao từ 6 động cơ độc lập mà không được phép làm gián đoạn luồng điều khiển cơ khí. Thử nghiệm được tiến hành bằng cách thao tác Joystick trên giao diện Web để cấp các mức ga khác nhau và quan sát sự phản hồi của Bảng giám sát.

Kết quả thực nghiệm:

Cụm Trái (Ga gốc: 98)		Cụm Phải (Ga gốc: 98)	
Bánh L1:	98 PWM 156 RPM	Bánh R1:	98 PWM 152 RPM
Bánh L2:	88 PWM 153 RPM	Bánh R2:	90 PWM 141 RPM
Bánh L3:	98 PWM 143 RPM	Bánh R3:	98 PWM 159 RPM

Hình 4.3.3 Giao diện Web hiển thị thời gian thực tốc độ (RPM) của 6 động cơ

Kết quả thực nghiệm:

- Đáp ứng thời gian thực: Khi người dùng đẩy thanh ga (Throttle) trên Joystick, biểu tượng trạng thái vận hành lập tức chuyển đổi (Ví dụ: FORWARD), đồng thời giá trị băm xung (PWM) và tốc độ vòng quay (RPM) của toàn bộ 6 bánh xe L1, L2, L3 và R1, R2, R3 được cập nhật đồng loạt với độ trễ gần như bằng không.
- Độ chính xác của thuật toán Động học: Thử nghiệm thao tác đánh lái (Steering) cho thấy rõ sự chênh lệch tốc độ giữa hai cụm bánh. Ví dụ: khi đánh lái nhẹ sang trái, cụm bánh Phải ghi nhận vận tốc (Ví dụ: 159 RPM) cao hơn rõ rệt so với cụm bánh Trái (Ví dụ: 143 RPM). Điều này chứng minh giải thuật Skid-Steer Mixing đã nội suy chính xác và Motor Driver đã đáp ứng đúng lệnh.
- Độ ổn định của hệ thống đa nhân: Ở dải tốc độ cao (PWM > 90), 6 cảm biến Hall gửi về hàng ngàn ngắt phần cứng mỗi giây. Tuy nhiên, nhờ cơ chế khóa găng (Spinlock) bảo vệ vùng nhớ và thuật toán M/T (Method of Time) hoạt động cô lập ở Core 0, dữ liệu RPM hiển thị rất ổn định, không xuất hiện hiện tượng giật lag giao diện hay mất kiểm soát xe.

Kết luận chung phần Giám sát: Hệ thống đã hoàn thành xuất sắc yêu cầu giám sát từ xa. Đường truyền song công WebSocket hoạt động ổn định, đảm bảo người dùng

có cái nhìn toàn cảnh về tình trạng điện năng và trạng thái động lực học của Robot theo thời gian thực.

CHƯƠNG 5: KẾT LUẬN

5.1. Các kết quả đạt được

Sau thời gian nghiêm túc nghiên cứu, thiết kế và thi công, đồ án “Thiết kế chương trình điều khiển và giám sát vận hành từ xa cho Robot địa hình 6x6” đã hoàn thành xuất sắc các mục tiêu đề ra ban đầu, cụ thể:

- Về phần cứng: Đã hoàn thiện thiết kế nguyên lý và gia công thành công hệ thống mạch in PCB (Bo mạch Điều khiển Trung tâm và Bo mạch Gia công Tín hiệu). Hệ thống điện hoạt động ổn định, đảm bảo cung cấp năng lượng an toàn và giao tiếp chống nhiễu tốt với các cơ cấu chấp hành (6 Motor Driver) và cảm biến.
- Về kiến trúc phần mềm: Đã xây dựng thành công kiến trúc phần mềm hướng đối tượng (OOP) trên nền tảng C++. Việc đóng gói các lớp phần cứng giúp mã nguồn dễ đọc, dễ bảo trì và có tính tái sử dụng cao.
- Về năng lực điều khiển thời gian thực: Áp dụng thành công hệ điều hành FreeRTOS trên cấu trúc đa nhân (Dual-Core) của ESP32. Đã chứng minh được bằng thực nghiệm khả năng cách ly hoàn toàn độ trễ mạng, khóa cứng chu kỳ điều khiển cơ khí ở mức chính xác 20ms (50Hz), đảm bảo tính thời gian thực (Hard Real-Time) khắt khe của hệ thống.
- Về giải thuật an toàn động học: Triển khai hoàn thiện giải thuật trộn xung lái trượt (Skid-Steer Mixing), bộ lọc khởi động mềm (Ramp-Step) và đặc biệt là Máy trạng thái (FSM) với tính năng phanh liên khóa (Anti-Shock Braking). Robot vận hành êm ái, chuyển hồi trạng thái mượt mà, bảo vệ tuyệt đối hộp số cơ khí khỏi hiện tượng sốc dòng và gãy vỡ do đảo chiều đột ngột.
- Về truyền thông và giám sát: Xây dựng thành công trạm giám sát đa nền tảng, tích hợp 3 nguồn điều khiển (RC, WebServer, ESP-NOW) với cơ chế phân quyền và bảo vệ rớt mạng (Timeout Failsafe) hoạt động hoàn hảo. Dữ liệu vận hành (Tốc độ 6 bánh xe, Điện áp pin) được thu thập bằng phương pháp M/T, đóng gói chuẩn JSON và hiển thị trực quan theo thời gian thực với độ trễ cực thấp.

5.2. Thuận lợi và khó khăn

- Thuận lợi: Sự hỗ trợ nhiệt tình từ Giảng viên hướng dẫn cùng nguồn tài liệu phong phú về nền tảng vi điều khiển ESP32 và hệ điều hành FreeRTOS đã giúp quá trình thiết kế giải thuật được rút ngắn đáng kể. Nền tảng cơ khí Rocker-

Bogie của Robot đã được gia công chắc chắn từ trước, tạo điều kiện thuận lợi để tập trung hoàn toàn vào khâu lập trình và mạch điện.

- Khó khăn: Việc xử lý nhiễu điện từ (EMI) phát sinh từ 6 động cơ công suất lớn tác động lên các tín hiệu analog (Đo pin) và xung vuông (Hall sensor) là một thách thức lớn. Ngoài ra, việc thiết kế cơ chế khóa găng (Spinlock) để chống "xé dữ liệu" giữa hai nhân CPU đòi hỏi thời gian dài nghiên cứu và thử nghiệm để hệ thống không bị treo (Deadlock).

5.3. Tồn tại và hạn chế

Mặc dù đã đạt được những kết quả khả quan, hệ thống vẫn còn một số điểm hạn chế nhất định:

- Giải thuật đo tốc độ M/T dù đã được nhân 3 độ phân giải bằng mạch cứng, nhưng khi xe di chuyển ở tốc độ cực kỳ chậm (crawling), số lượng xung thu về trong chu kỳ 350ms vẫn còn ít, có thể gây ra sai số nhỏ.
- Hệ thống hiện tại chỉ đang vận hành dựa trên tín hiệu điều khiển trực tiếp từ người dùng (Open-loop control về mặt hướng đi) mà chưa có khả năng tự hiệu chỉnh quỹ đạo bằng cảm biến góc nghiêng hay la bàn từ.
- Giao diện WebServer lưu trữ trên bộ nhớ SPIFFS đôi khi gặp tình trạng phản hồi chậm nếu có quá nhiều thiết bị truy cập đồng thời vào mạng SoftAP.

5.4. Quá trình đúc kết kinh nghiệm

Thông qua quá trình thực hiện đồ án, bản thân sinh viên đã đúc kết được nhiều bài học kỹ thuật quý giá:

- Nâng cao tư duy thiết kế hệ thống Nhúng (Embedded Systems), chuyển từ việc viết code chạy tuần tự (Super-loop) sang tư duy thiết kế Hệ điều hành thời gian thực (RTOS).
- Nắm vững kỹ năng vẽ mạch, tính toán linh kiện bảo vệ, phân tích nhiễu phân cứng và triển khai phần mềm bám sát với sơ đồ mạch in PCB thực tế.

5.5. Kiến nghị và Hướng phát triển

Từ những tồn tại trên, đề tài mở ra nhiều định hướng phát triển đầy tiềm năng trong tương lai để biến hệ thống thành một Robot tự hành (Autonomous Rover) hoàn chỉnh:

- Tích hợp cảm biến IMU (MPU6050/BNO085) và GPS: Cung cấp thông tin tọa độ và góc Yaw, Roll, Pitch để phát triển bộ điều khiển PID khép kín, giúp xe tự động đi thẳng bất chấp địa hình gồ ghề và có khả năng tự hành theo tọa độ (Waypoint Navigation).
- Nâng cấp chuẩn giao tiếp phần cứng: Thay thế chuẩn giao tiếp PWM hiện tại bằng chuẩn mạng CAN Bus giữa ESP32 và các Motor Driver để tăng cường khả năng chống nhiễu và đọc dữ liệu hồi tiếp hai chiều với tốc độ cao hơn.
- Tích hợp hệ thống Camera FPV: Bổ sung module xử lý ảnh (như Raspberry Pi hoặc ESP32-CAM) để truyền phát video trực tiếp (Video Streaming) về giao diện Web, hỗ trợ người điều khiển vận hành xe an toàn khi bị khuất tầm nhìn.